A Course End Project Report on

NEUROVOX

Voice-Based Parkinson's Disease Screening using Java

Submitted in partial fulfilment of the requirements of the course

JAVA PROGRAMMING (22ITC02N)

Bachelor of Engineering — III Semester

Computer Science and Engineering (IoT and Cyber Security including Blockchain Technology)

Submitted by

Pranav	160125749063
Bhanuteja	160125749048
Md. Ali	160125749055

Under the guidance of

Mrs. Sujatha Guptha

Faculty, Java Programming

DEPARTMENT OF COMPUTER ENGINEERING AND TECHNOLOGY

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)

Gandipet, Hyderabad – 500 075, Telangana

2026 – 2027

CERTIFICATE

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)

Department of Computer Engineering and Technology

This is to certify that the course end project report entitled **"Neurovox: Voice-Based Parkinson's Disease Screening using Java"** is a bonafide record of the work carried out by **Pranav (160125749063)**, **Bhanuteja (160125749048)** and **Md. Ali (160125749055)**, students of B.E. III Semester, CSE (IoT and Cyber Security including Blockchain Technology), in partial fulfilment of the requirements of the course **Java Programming (22ITC02N)** during the academic year 2026–2027.

The work embodied in this report has not been submitted elsewhere for any other course or degree.

Mrs. Sujatha Guptha
Course Faculty, Java Programming

Head of the Department
Computer Engineering and Technology

DECLARATION

We hereby declare that the project report entitled **“Neurovox: Voice-Based Parkinson’s Disease Screening using Java”**, submitted for the course Java Programming (22ITC02N), is a record of our own work carried out under the guidance of Mrs. Sujatha Guptha. The results in this report have been obtained by us from our own code and validation scripts, and the sources we used are acknowledged in the references.

Software assistance (an AI coding assistant) was used during development and for drafting parts of this report. Every algorithm and line of code in the submitted version has been reviewed, tested and can be explained by the team.

Place: Hyderabad
Date:

Pranav (160125749063)
Bhanuteja (160125749048)
Md. Ali (160125749055)

ACKNOWLEDGEMENT

We express our sincere gratitude to **Mrs. Sujatha Guptha**, our Java Programming faculty, for her guidance, for encouraging us to take up a project that connects programming with a real health problem, and for pushing us to understand and write every part of the code ourselves.

We thank the **Head of the Department of Computer Engineering and Technology** and the management of **Chaitanya Bharathi Institute of Technology** for providing the facilities and the opportunity to carry out this project.

We are grateful to the researchers who made their work openly available — in particular Max Little and colleagues for the UCI Parkinson's dataset, and Paul Boersma and David Weenink for Praat, which served as our reference throughout validation.

Finally, we thank our families and friends for their support and patience during the semester.

ABSTRACT

Parkinson’s disease affects more than 8.5 million people worldwide and is diagnosed clinically, often after much of the affected brain tissue has been lost. Changes in the voice — irregular vocal-fold vibration, unstable loudness and breathiness — appear in about nine out of ten patients and can be measured from a few seconds of sustained “aaah”.

Neurovox is a Java application that records or loads such a vowel and measures four classical voice properties with algorithms written from first principles: the fundamental frequency (F0) by normalised autocorrelation, and, after tracking every vocal cycle by cross-correlation with parabolic refinement, the cycle-to-cycle variation in timing (**jitter**), in loudness (**shimmer**) and the harmonics-to-noise ratio (**HNR**). Each measure is compared with a published reference limit (jitter 1.04%, shimmer 3.81%, HNR 20 dB) and a binary screening indication is given: *further check suggested* when two or more limits are exceeded, otherwise *no concern indicated*. The program never claims to diagnose.

The submitted version is a console program of about 395 lines in four short files — one `Main` class and two packages (`audio`, `analysis`) — with no external libraries. It is designed to be written by hand in an examination and uses the topics of the Java Programming syllabus: abstract classes, inheritance, nested classes, interfaces, packages, user-defined exceptions, multithreading with synchronisation, generics, collections, lambda expressions and buffered file I/O.

On 40 synthetic vowels with exactly known values, the program measured F0 with a mean error of 0.22 Hz (r = 1.000) and jitter with r = 0.92, and its final decision agreed with the reference decision on 33 of 40 files without any missed case. An earlier web prototype by the same team matched Praat to within 0.15% on 16 measures, but showed that a classifier trained on the public UCI dataset falls below the majority baseline (69.2% against 75.4%) when tested on unseen people — which is why Neurovox uses transparent limits instead.

Keywords: Parkinson’s disease, voice analysis, jitter, shimmer, harmonics-to-noise ratio, autocorrelation, Java, object-oriented programming.

TABLE OF CONTENTS

Certificate i
Declaration ii
Acknowledgement iii
Abstract iv
Table of contents v
List of figures vii
List of tables viii
List of abbreviations ix
1. INTRODUCTION 1
1.1 Background 1
1.2 Motivation 1
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Scope and Non-Goals 2
1.6 Organisation of the Report 3
2. LITERATURE SURVEY 4
2.1 Voice and Speech in Parkinson's Disease 4
2.2 Acoustic Measures: Jitter, Shimmer and HNR 4
2.3 Pitch Detection Algorithms 5
2.4 Machine Learning on Voice Measures 5
2.5 Summary and Research Gap 5
3. SYSTEM REQUIREMENTS AND ANALYSIS 7
3.1 Functional Requirements 7
3.2 Non-Functional Requirements 7
3.3 Hardware and Software Requirements 7
3.4 Feasibility Study 8
3.5 Use Case Diagram 8
4. SYSTEM DESIGN 9
4.1 System Architecture 9
4.2 Data Flow 9
4.3 Package Structure 10
4.4 Class Design 11
4.5 Thread Design 11
4.6 Exception Design 12
4.7 File Formats 12
5. ALGORITHMS AND SIGNAL PROCESSING 13
5.1 Digital Audio Basics 13
5.2 Pre-processing 13
5.3 Pitch Detection by Autocorrelation 13
5.4 Cycle-by-Cycle Tracking 14

5.5 Jitter, Shimmer and F0 14
5.6 Harmonics-to-Noise Ratio 15
5.7 Decision Rule 15
5.8 Synthetic Demo Voice 15
5.9 Complexity Analysis 16
6. IMPLEMENTATION 17
6.1 Development Environment 17
6.2 Package audio — class VoiceSource 17
6.3 Package analysis — classes PitchTracker and VoiceReport 18
6.4 Class Main 19
6.5 Mapping to the Java Programming Syllabus (22ITC02N) 19
6.6 Compiling and Running 20
7. TESTING AND RESULTS 21
7.1 Testing Strategy 21
7.2 Test Cases 21
7.3 Output Screens 22
7.4 Accuracy on Test Vowels with Known Values 23
7.5 Agreement of the Final Decision 25
7.6 Performance 25
8. EXTENDED WEB PROTOTYPE AND RESEARCH FINDINGS 26
8.1 Overview 26
8.2 Java Versus Praat 27
8.3 Autocorrelation Versus YIN 28
8.4 The Research Model and the Go/No-Go Decision 29
8.5 Consequences for the Console Version 29
9. LIMITATIONS, ETHICS AND FUTURE SCOPE 30
9.1 Limitations 30
9.2 Ethical Considerations 30
9.3 Future Scope 30
10. CONCLUSION 31
REFERENCES 32
APPENDIX A: SOURCE CODE 34
A.1 Main.java (default package) 34
A.2 audio/VoiceSource.java (package audio) 36
A.3 analysis/PitchTracker.java (package analysis) 38
A.4 analysis/VoiceReport.java (package analysis) 40
APPENDIX B: USER MANUAL 42
APPENDIX C: TEAM CONTRIBUTION 43

LIST OF FIGURES

Figure 3.1: Use case diagram of the console application....8
Figure 4.1: Architecture of the console version: packages, classes and files....9
Figure 4.2: Data flow from voice input to stored result....10
Figure 4.3: Class diagram (UML)....11
Figure 4.4: Life cycle of the analysis thread and the main thread....12
Figure 5.1: Left: 40 ms of a test vowel with the detected period marked. Right: correlation r for each candidate pitch....14
Figure 5.2: 60 ms of the steady and shaky demo voices....16
Figure 7.1: Compiling and starting the program; the main menu....22
Figure 7.2: Result for the steady demo voice: no marker outside the limits....22
Figure 7.3: Result for the shaky demo voice: all three markers outside the limits....22
Figure 7.4: Session comparison (sorted by jitter) and the saved history file....23
Figure 7.5: Error handling: no microphone, noise, silence, wrong format and invalid menu input....23
Figure 7.6: Console version against the known true values for 40 test vowels (dashed line = perfect agreement)....24
Figure 7.7: Measured HNR against the amount of added noise....24
Figure 8.1: Web prototype: recording the sustained vowel; the orb reacts to the voice….26
Figure 8.2: Web prototype: top of the measurement report, with recording quality....27
Figure 8.3: Web prototype: history page with the trend of one measurement....27
Figure 8.4: Full Java implementation against Praat for each measurement....28
Figure 8.5: Pitch error and speed of the two detectors....29

LIST OF TABLES

Table 2.1: Voice signs of Parkinson’s disease and how they can be measured....4
Table 2.2: Summary of related work....5
Table 3.1: Functional requirements....7
Table 3.2: Hardware and software requirements....7
Table 4.1: Packages and their responsibilities....10
Table 4.2: Exceptions and the messages shown to the user....12
Table 5.1: Reference limits used for the screening decision....15
Table 6.1: Syllabus topics and where they are used....19
Table 7.1: Functional test cases....21
Table 7.2: Accuracy of the console version on 40 test vowels....24
Table 7.3: Shimmer over-reading against the noise level....25
Table 7.4: Decision agreement on 40 test vowels....25
Table 8.1: Full Java implementation against Praat on 40 vowels (selected measures)....28
Table 8.2: Pitch detector comparison (40 vowels)....28
Table 8.3: Research model evaluation (positive class = Parkinson’s)....29
Table C.1: Work division by package....43

LIST OF ABBREVIATIONS

API	Application Programming Interface
CSV	Comma-Separated Values
DC	Direct Current (constant offset)
F0	Fundamental frequency (pitch) of the voice
HNR	Harmonics-to-Noise Ratio
JDK	Java Development Kit
MAE	Mean Absolute Error
MDVP	Multi-Dimensional Voice Program (Kay Elemetrics)
MFCC	Mel-Frequency Cepstral Coefficients
NHR	Noise-to-Harmonics Ratio
PCM	Pulse-Code Modulation
PD	Parkinson’s Disease
RAP	Relative Average Perturbation
RMS	Root Mean Square
ROC-AUC	Area Under the Receiver Operating Characteristic curve
SNR	Signal-to-Noise Ratio
UCI	University of California, Irvine (Machine Learning Repository)
UPDRS	Unified Parkinson’s Disease Rating Scale
WAV	Waveform Audio File Format
YIN	Pitch-detection algorithm of de Cheveigné and Kawahara

CHAPTER 1

INTRODUCTION

1.1 Background

Parkinson's disease (PD) is a progressive disorder of the nervous system in which the dopamine-producing neurons of the substantia nigra are gradually lost. It is best known for its motor signs — tremor at rest, slowness of movement (bradykinesia), rigidity and unstable posture — but it also affects sleep, mood, smell and speech. The World Health Organization estimates that more than 8.5 million people were living with Parkinson's disease in 2019, and that disability and deaths due to PD are increasing faster than for any other neurological disorder [1].

There is no blood test or routine scan that confirms Parkinson's disease in a living patient. Diagnosis is clinical: a neurologist examines the patient, takes a history and rules out conditions that look similar, using criteria such as those of the International Parkinson and Movement Disorder Society [2]. By the time the classic tremor appears, a large share of the relevant neurons has already been lost, so any cheap, non-invasive signal that could prompt an earlier visit to a specialist is valuable.

Speech is one such signal. The same loss of motor control that slows the hands also affects the muscles of the larynx, tongue and lips. In a classic study of 200 patients, Logemann and colleagues found voice (laryngeal) problems in 89% of them [3], and later acoustic studies showed measurable changes even in early, untreated patients [4]. Typical changes include a softer voice (hypophonia), reduced pitch variation (monopitch), a breathy or hoarse quality and irregular vibration of the vocal folds. Many of these changes are too small for a listener to notice, but they can be computed from a recording.

1.2 Motivation

A sustained vowel — simply saying "aaah" for a few seconds — is the standard recording used in voice research because it removes the effect of words and accent and leaves only the vibration of the vocal folds. From such a recording, three classical measures can be computed: **jitter** (how much the length of each vocal cycle varies), **shimmer** (how much the loudness of each cycle varies) and the **harmonics-to-noise ratio (HNR)** (how much of the sound is regular voice rather than breath noise). Published work has repeatedly linked these measures to Parkinson's disease [5], [6].

Most tools that compute these measures — Praat [7], the Multi-Dimensional Voice Program (MDVP) [8] — are specialist software. The motivation for Neurovox was to show that the complete chain, from raw microphone bytes to a screening indication, can be

written and explained in plain Java by second-year students, without hiding the core work inside an audio library.

1.3 Problem Statement

Design and implement a Java application that takes a short sustained-vowel recording (live from a microphone or from a WAV file), measures the pitch, jitter, shimmer and harmonics-to-noise ratio of the voice using algorithms written in Java, compares them with published reference limits, and reports a **binary screening indication** — "further check suggested" or "no concern indicated" — together with the measurements, while keeping a history of results. The application must be small enough to be written by hand in an examination, must use packages, and must not claim to diagnose the disease.

1.4 Objectives

1. To capture a voice sample from the microphone (Java Sound API), from a 16-bit WAV file, or from a built-in synthetic voice for demonstration.
2. To implement pitch detection (autocorrelation) and cycle-by-cycle tracking (cross-correlation) in Java from first principles.
3. To compute jitter, shimmer and HNR and compare each with a published reference limit.
4. To produce a clear binary screening result with the measurements and the limits used.
5. To demonstrate the object-oriented concepts of the Java Programming course (22ITC02N): classes, inheritance, abstract classes, interfaces, inner classes, packages, exceptions, multithreading, generics, collections and file I/O.
6. To validate the measurements against recordings whose true values are known.

1.5 Scope and Non-Goals

Neurovox is deliberately limited to **binary screening**. The following are explicitly out of scope:

- **Diagnosis.** The program never states that a person has or does not have Parkinson's disease. A flagged voice only means "see a doctor"; many other conditions (a cold, smoking, vocal strain, age) change the same measures.
- **Disease staging** (for example the Hoehn and Yahr scale). Published error margins are too wide to separate neighbouring stages, so a stage would suggest a precision the signal does not have.
- **Treatment advice.** This is medical practice and depends on information not present in a voice recording.

The project exists in two forms. The **exam (console) version**, described in Chapters 3 to 7, is the subject of this report: one `Main` class and two packages in four short files (about 395 lines), with no external libraries. The **extended web prototype**, built earlier in the semester and summarised in Chapter 8, adds a browser interface, a database and a research classifier; its validation results explain several design decisions of the console version.

1.6 Organisation of the Report

Chapter 2 reviews the literature on voice changes in Parkinson's disease and on the measures used. Chapter 3 lists the requirements, and Chapter 4 presents the design: architecture, packages, classes, threads and exceptions. Chapter 5 explains the signal-processing algorithms and their formulas. Chapter 6 walks through the implementation and maps it to the course syllabus. Chapter 7 presents the test cases, output screens and accuracy results. Chapter 8 summarises the extended web prototype and its research findings. Chapter 9 discusses limitations, ethics and future work, and Chapter 10 concludes. The complete source code is given in Appendix A.

CHAPTER 2

LITERATURE SURVEY

2.1 Voice and Speech in Parkinson's Disease

Speech problems in Parkinson's disease are grouped under the term *hypokinetic dysarthria*. Logemann et al. [3] examined 200 patients and found laryngeal (voice) dysfunction in 89%, often as the first speech sign to appear. Rusz et al. [4] studied 46 Czech speakers — 23 with newly diagnosed, untreated Parkinson's disease and 23 healthy controls — and found that voice and prosody measures already differed in this early group, with reduced pitch variation being one of the most sensitive signs. These findings support the use of a sustained vowel as a simple, language-independent recording task.

Table 2.1: *Voice signs of Parkinson's disease and how they can be measured*

Clinical sign	What happens	Acoustic measure
Hypophonia	Softer, unstable loudness	Shimmer, intensity
Vocal-fold rigidity	Irregular cycle-to-cycle timing	Jitter
Incomplete closure of the glottis	Breathiness, added noise	HNR / NHR
Monopitch	Narrow pitch range	F0 range, F0 standard deviation
Imprecise articulation	Blurred consonants, slower syllables	MFCC, syllable rate (web prototype only)

2.2 Acoustic Measures: Jitter, Shimmer and HNR

Teixeira et al. [6] review the definitions of the classical perturbation measures and their use in detecting voice pathology. **Local jitter** is the average absolute difference between consecutive cycle lengths divided by the average cycle length; **local shimmer** is the same quantity computed on the peak amplitude of each cycle. Kay Elemetrics' MDVP [8] — the program whose output columns appear in the UCI Parkinson's dataset — suggests thresholds of 1.04% for jitter and 3.81% for shimmer, above which a voice is considered potentially pathological. A harmonics-to-noise ratio below about 20 dB is commonly taken to indicate a hoarse or breathy voice [6].

Boersma [9] showed that both the fundamental frequency and the harmonics-to-noise ratio can be estimated accurately from the normalised autocorrelation of the signal. If r is the correlation between the signal and itself shifted by one period, the harmonic part of the energy is r and the noise part is $1 - r$, giving $\mathrm{HNR} = 10 \cdot \log_{10}(r / (1 - r))$. This is the method used by Praat [7] and by Neurovox.

2.3 Pitch Detection Algorithms

The fundamental frequency (F0) is the rate at which the vocal folds vibrate, typically 85–180 Hz for adult men and 165–255 Hz for adult women. The **autocorrelation** method [9] compares the signal with delayed copies of itself and selects the delay at which they match best. The **YIN** algorithm of de Cheveigné and Kawahara [10] uses a cumulative-mean-normalised difference function and is much faster. In the extended prototype (Chapter 8) both were implemented and compared on 40 test vowels: autocorrelation had a mean error of 0.77 Hz and no octave errors on clean audio, while YIN had 3.25 Hz and 1.8% octave errors, and the gap widened with added noise. The console version therefore uses autocorrelation.

2.4 Machine Learning on Voice Measures

Little et al. [11] introduced nonlinear voice measures (recurrence period density entropy and detrended fluctuation analysis) for detecting general voice disorders. They then applied these together with the classical MDVP measures to Parkinson's disease [5]: 195 sustained vowels from 31 people (23 with Parkinson's disease), 22 measures per recording, published as the widely used **UCI Parkinson's dataset** [12]. Using a support-vector machine they reported 91.4% accuracy and argued that such dysphonia measurements are suitable for remote monitoring. Tsanas et al. [13] used 5,875 recordings from 42 patients to estimate the UPDRS symptom score remotely, and Sakar et al. [14] compared many signal-processing feature sets for PD classification.

An important caution comes from how these models are tested. Each person in the UCI dataset contributes about six recordings. If recordings are split randomly into training and test sets, the same person appears in both, and the model partly learns to recognise *voices* rather than *disease*. When the extended prototype retrained a logistic-regression model with the split grouped by person, accuracy fell from 80.5% (random split) to 69.2% — below the 75.4% obtained by always predicting the majority class (Section 8.5). This finding is the main reason the console version uses transparent reference limits instead of a trained classifier.

2.5 Summary and Research Gap

Table 2.2: *Summary of related work*

Work	Data	Method	Key result
Little et al. 2007 [11]	General voice-disorder recordings	Nonlinear measures (RPDE, DFA)	New measures for disordered voices
Little et al. 2009 [5]	195 vowels, 31 people (UCI)	Feature selection + SVM	91.4% accuracy (recording-level split)
Tsanas et al. 2010 [13]	5,875 recordings, 42 patients	Regression on voice measures	UPDRS estimated within about 7.5 points

Work	Data	Method	Key result
Rusz et al. 2011 [4]	23 early PD, 23 controls	Acoustic analysis of several tasks	Changes present before treatment
Boersma 1993 [9]	Synthetic and real voices	Autocorrelation pitch and HNR	Basis of Praat's method
de Cheveigné & Kawahara 2002 [10]	Speech databases	YIN pitch detector	Low error, fast
This project	40 vowels with known truth	Java autocorrelation + cross-correlation	F0 error 0.22 Hz; jitter r = 0.92

The gap this project addresses is not a new detection method but a **transparent, explainable implementation**: every number in the report is computed by Java code that the authors wrote and can explain line by line, and every number is checked against recordings whose correct answer is known.

CHAPTER 3

SYSTEM REQUIREMENTS AND ANALYSIS

3.1 Functional Requirements

Table 3.1: *Functional requirements*

ID	Requirement
FR1	Ask for the patient's name and show a menu until the user chooses Exit.
FR2	Record 5 seconds of voice from the microphone and save it as recording.wav.
FR3	Read a 16-bit PCM WAV file (mono or stereo, any sample rate).
FR4	Generate a steady or a shaky synthetic voice for demonstration without a microphone.
FR5	Reject recordings that are too short, too quiet or contain no clear voice, with a message saying what to change.
FR6	Measure F0, jitter, shimmer and HNR of a sustained vowel.
FR7	Compare each measure with its reference limit and report the binary result.
FR8	Append every result to history.csv and show the saved history on request.
FR9	Compare all voices analysed in the current session, steadiest first.

3.2 Non-Functional Requirements

- **Simplicity:** only the Java standard library; compiles with a single `javac` command on JDK 11 or newer.
- **Writability:** the core (Main, VoiceSource, PitchTracker, VoiceReport) must fit in about 250 lines, and no file may be longer than about 120 lines, so it can be written by hand in three hours.
- **Responsiveness:** analysis of a 5-second recording completes in under one second on a laptop.
- **Robustness:** no input — wrong menu choice, missing file, noise, silence — may crash the program.
- **Honesty:** results must state the limits used and must not use the word "diagnosis".
- **Privacy:** all data stays on the computer; nothing is sent over a network.

3.3 Hardware and Software Requirements

Table 3.2: *Hardware and software requirements*

Item	Minimum	Used in development
Processor	Any x86-64 or ARM64	Apple M4 / Intel Core i5
Memory	2 GB RAM	8–16 GB
Microphone	Optional (demo works without)	Built-in laptop microphone

Item	Minimum	Used in development
Operating system	Windows, Linux or macOS	macOS 15, Ubuntu 24.04, Windows 11
Java	JDK 11 or newer	JDK 17 and 21
Editor	Any text editor	VS Code, IntelliJ IDEA
Validation tools	—	Python 3, Praat (via Parselmouth)

3.4 Feasibility Study

Technical feasibility. The Java Sound API (`javax.sound.sampled`) is part of the JDK and can both record from a microphone and read WAV files, so no library is needed. The algorithms need only arrays, loops and the `Math` class. **Operational feasibility.** The program is a text menu that runs in any command prompt; options 3 and 4 need no microphone, which matters in a college lab. **Economic feasibility.** All tools used are free; the cost is limited to development time.

3.5 Use Case Diagram

Figure 3.1 shows the actions available to the user. Every way of obtaining a voice — recording, loading a file or running a demo — leads to the same screening report.

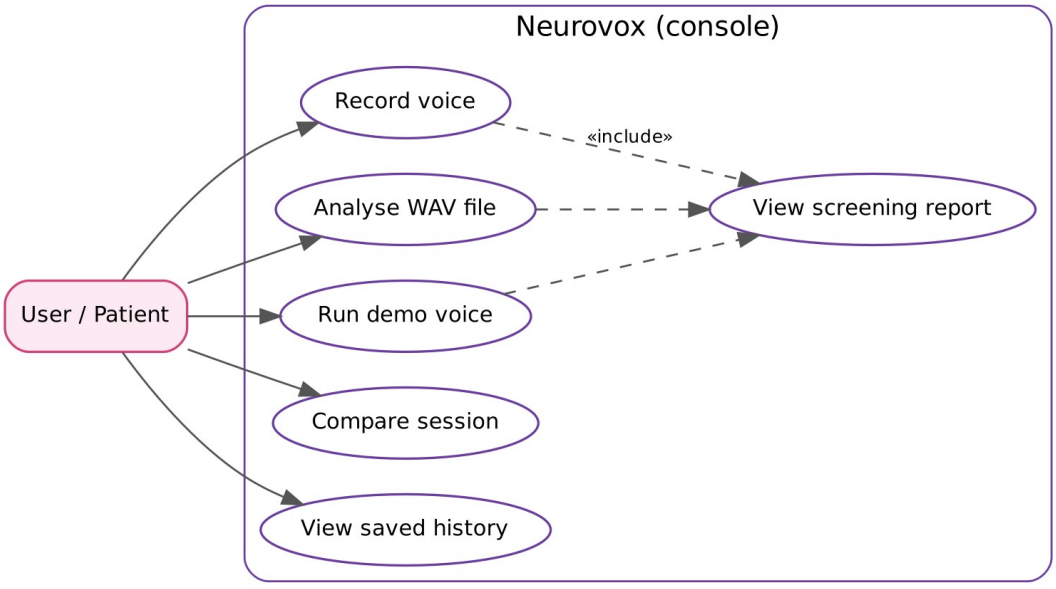


Figure 3.1: *Use case diagram of the console application*

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The application is divided into three parts (Figure 4.1). The **audio** package knows where a voice comes from; the **analysis** package knows how to measure it; the `Main` class in the default package handles the user, runs the analysis on a separate thread and keeps the history file. Neither package depends on `Main`, so each can be tested or reused on its own.

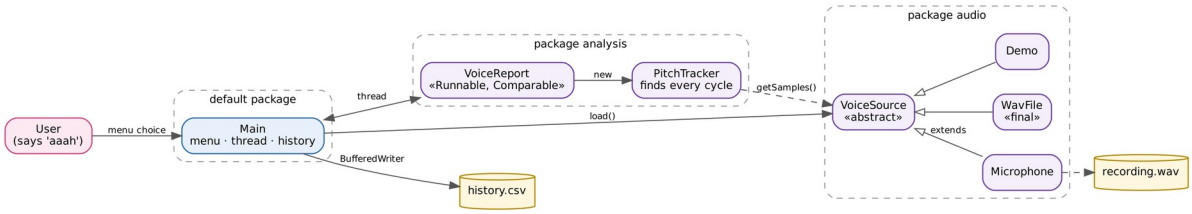


Figure 4.1: *Architecture of the console version: packages, classes and files*

4.2 Data Flow

Figure 4.2 shows the path of one recording from input to result. Steps 2 and 3 belong to the audio package and the start of the analysis; steps 3 to 5 are in `PitchTracker`, steps 6 to 8 in `VoiceReport`, and step 9 is in `Main`.

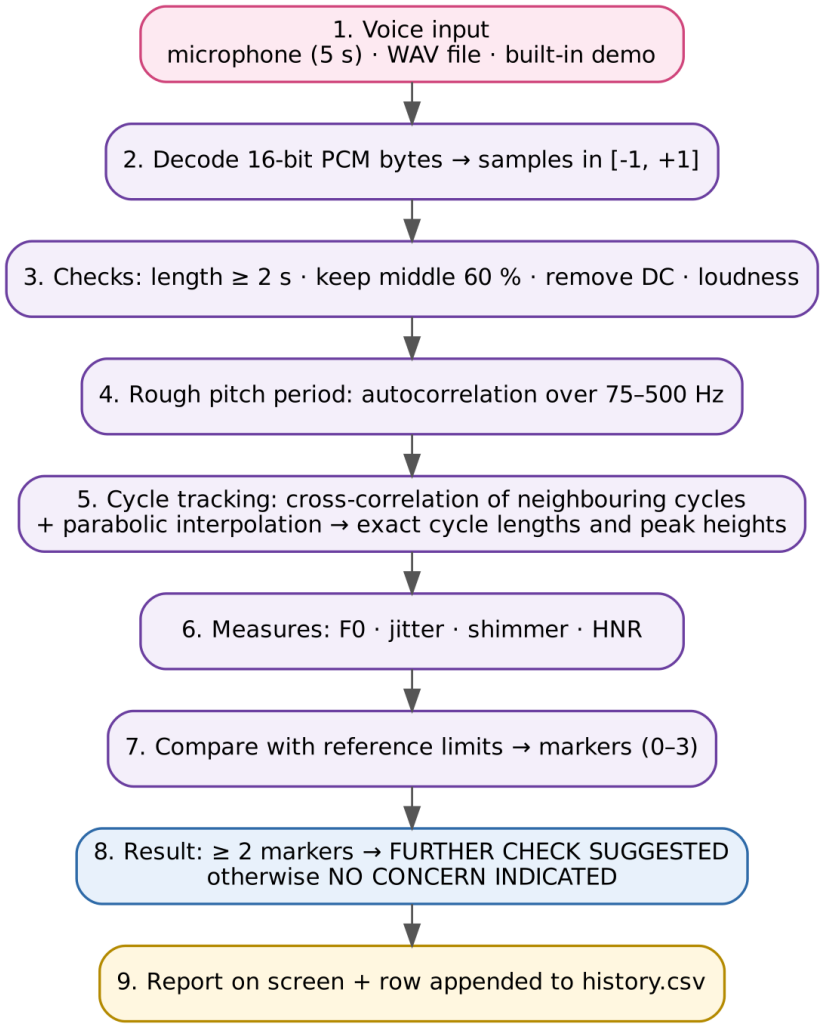


Figure 4.2: *Data flow from voice input to stored result*

4.3 Package Structure

Table 4.1: *Packages and their responsibilities*

Package	File	Responsibility
audio	VoiceSource.java	Abstract voice source; three subclasses (Microphone, WavFile, Demo); byte-to-sample conversion; the VoiceException class
analysis	PitchTracker.java	Pre-processing checks, rough pitch by autocorrelation, cycle-by-cycle tracking; gives cycle lengths, peak heights and similarity
analysis	VoiceReport.java	F0, jitter, shimmer, HNR, reference limits, report text, sorting by jitter; runs on a thread
(default)	Main.java	Menu, thread management, session list, history file, comparison

Packages group related classes, prevent name clashes and control access. The default package can import named packages, but not the reverse, which is why the reusable code is in `audio` and `analysis` while only `Main` stays in the default package. Because the folders

`audio/` and `analysis/` sit next to `Main.java`, the current directory is the CLASSPATH root and no `-cp` option is needed.

4.4 Class Design

Figure 4.3 is the class diagram. `VoiceSource` is abstract: it holds the samples and sample rate and declares the abstract method `load()`. The three subclasses override `load()` in different ways, so `Main` can hold any of them in a `VoiceSource` variable and call `voice.load()` — the correct version is chosen at run time (dynamic method dispatch). `WavFile` is declared `final` to show how inheritance can be prevented. The analysis is split into two small classes. `PitchTracker` does the signal processing in its constructor and exposes its results (`periods`, `peaks`, `similarity`) with *default (package-private) access*, so only classes in the `analysis` package can read them. `VoiceReport` turns those results into the four measures and the decision; it implements `Runnable`, so that it can run on its own thread, and `Comparable`, so that a list of results can be sorted by jitter.

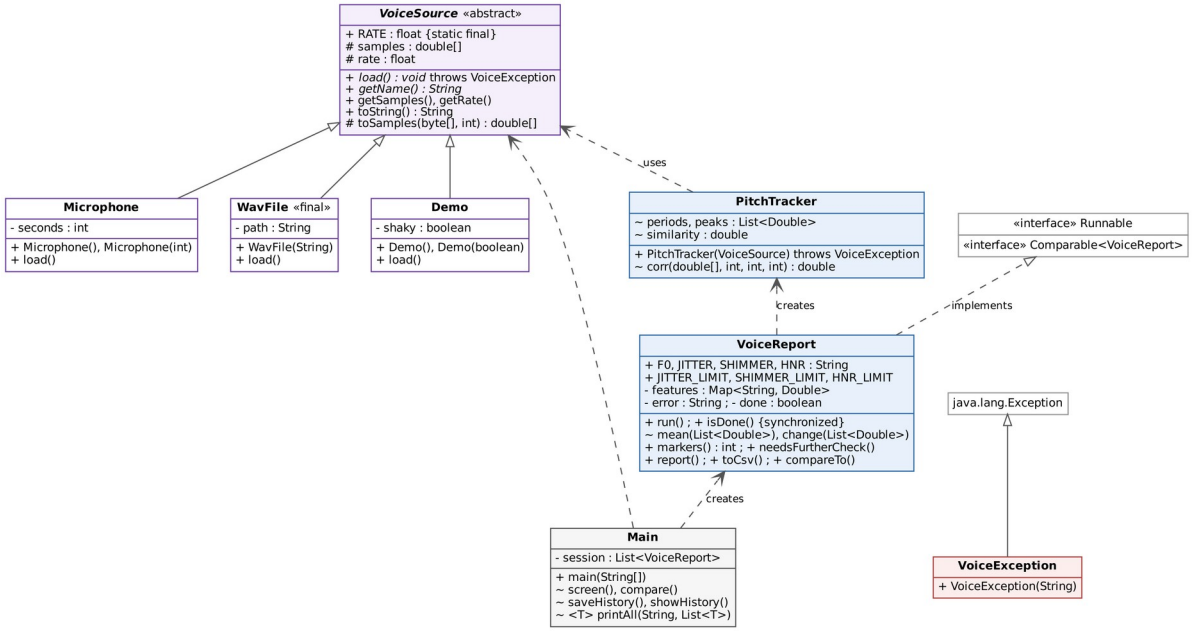


Figure 4.3: *Class diagram (UML)*

4.5 Thread Design

Analysis is run on a separate thread so that the main thread stays free to show progress (Figure 4.4). `Main` creates `new Thread(analyzer)` and calls `start()`. While the worker runs, the main thread prints a dot every 50 ms and checks `isDone()`. Both `isDone()` and `finish()` are `synchronized`, so the flag written by the worker is guaranteed to be visible to the main thread. Finally `join()` waits for the worker to terminate before the report is read.

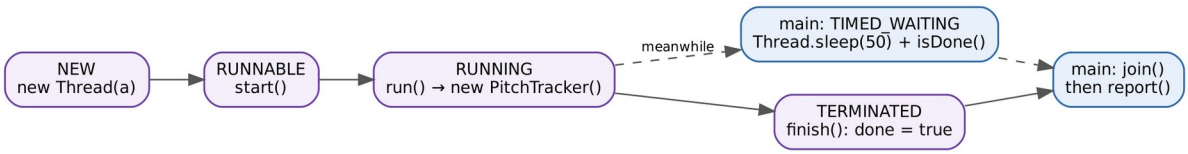


Figure 4.4: *Life cycle of the analysis thread and the main thread*

An exception thrown inside `run()` cannot propagate to the main thread, because `run()` is not allowed to declare checked exceptions. The worker therefore stores the message in the field `error`; when the main thread later calls `report()`, that method re-throws it as a `VoiceException`. The `finally` block guarantees that `done` is set even if analysis fails, so the progress loop always ends.

4.6 Exception Design

Table 4.2: *Exceptions and the messages shown to the user*

Situation	Where detected	Message
No microphone or access denied	Microphone.load()	Microphone not available: …
File missing or not a WAV	WavFile.load()	Cannot read <file>: …
WAV not 16-bit PCM	WavFile.load()	Please use a 16-bit PCM WAV file
Recording shorter than 2 s	PitchTracker()	Too short. Record at least 2 seconds.
RMS level below 0.01	PitchTracker()	Too quiet. Say 'aah' louder.
Fewer than 10 cycles found	PitchTracker()	No steady voice. Hold one 'aah' for 3–5 s.
Mean cycle similarity below 0.5	PitchTracker()	No clear voice found (only noise)…
Any unexpected runtime error	run()	Analysis failed: …
Menu input not 0–6	main()	Invalid choice.

All recoverable problems are reported through one user-defined checked exception, `VoiceException`, declared as a static nested class of `VoiceSource`. Using a checked exception forces every caller either to handle it or to declare it with `throws`, so no failure can be silently ignored.

4.7 File Formats

recording.wav is a standard RIFF WAV file (44,100 samples per second, 16-bit, mono) written by `AudioSystem.write`, so it can be opened in Praat or any audio player. **history.csv** is a plain text file with one line per analysis, in the order: name, date and time, F0 (Hz), jitter (%), shimmer (%), HNR (dB), result. For example:

```
Pranav,Mon Oct 26 13:45:35 IST 2026,139.9,0.30,2.35,35.1,OK
```

CHAPTER 5

ALGORITHMS AND SIGNAL PROCESSING

5.1 Digital Audio Basics

A microphone produces a voltage that follows the air pressure. The sound card measures this voltage 44,100 times per second (the **sample rate**) and stores each measurement as a 16-bit signed integer between −32,768 and 32,767 (**PCM**, pulse-code modulation). A five-second recording is therefore 220,500 numbers, or 441,000 bytes. In a WAV file each sample is stored *little-endian*: the low byte first, then the high byte. The method `toSamples()` rebuilds each sample and scales it to the range −1 to +1:

$$x[i] = ((b[2i+1] \ll 8) \mid (b[2i] \,\&\, 0xFF)) \,/\, 32768 \tag{5.1}$$

The mask `& 0xFF` is needed because Java's `byte` is signed: without it, a low byte such as 0xF0 would be sign-extended to a negative number and corrupt the sample. For stereo files only the first channel is used, by stepping 2 × channels bytes per sample.

5.2 Pre-processing

Three steps prepare the signal. (1) The first and last 20% of the recording are dropped, because the start and end of a sustained vowel are unstable. (2) The mean value is subtracted from every sample to remove any DC offset introduced by the microphone. (3) The root-mean-square (RMS) level is computed; if it is below 0.01 (about −40 dB below full scale), the recording is rejected as too quiet.

$$RMS = \sqrt{\,(1/N)\,\Sigma\, x[n]^2\,} \tag{5.2}$$

5.3 Pitch Detection by Autocorrelation

A voiced sound repeats itself once per vocal cycle. If the signal is compared with a copy of itself shifted by τ samples, the match is best when τ equals one period. The normalised correlation between two segments of length L starting at positions a and b is:

$$r(a,\,b,\,L) = \Sigma\, x[a+i]\cdot x[b+i] \,/\, \sqrt{\,\Sigma\, x[a+i]^2 \cdot \Sigma\, x[b+i]^2\,}, \quad i = 0 \ldots L-1 \tag{5.3}$$

r is 1 when the segments are identical and near 0 when they are unrelated. To find the rough period, `PitchTracker` computes r(0, τ, L) for every lag τ between the sample rate ÷ 500 and the sample rate ÷ 75 — that is, for every pitch between 75 Hz and 500 Hz — and keeps the τ with the highest r. Figure 5.1 shows this on a test vowel: the curve peaks at a lag of 295 samples, which is 44,100 ÷ 295 = 149.5 Hz. Note the second-highest peak near 75 Hz (twice the period) with r ≈ 0.9: choosing it would be an *octave error*, which is why the method selects the global maximum and the search range stops at 75 Hz.

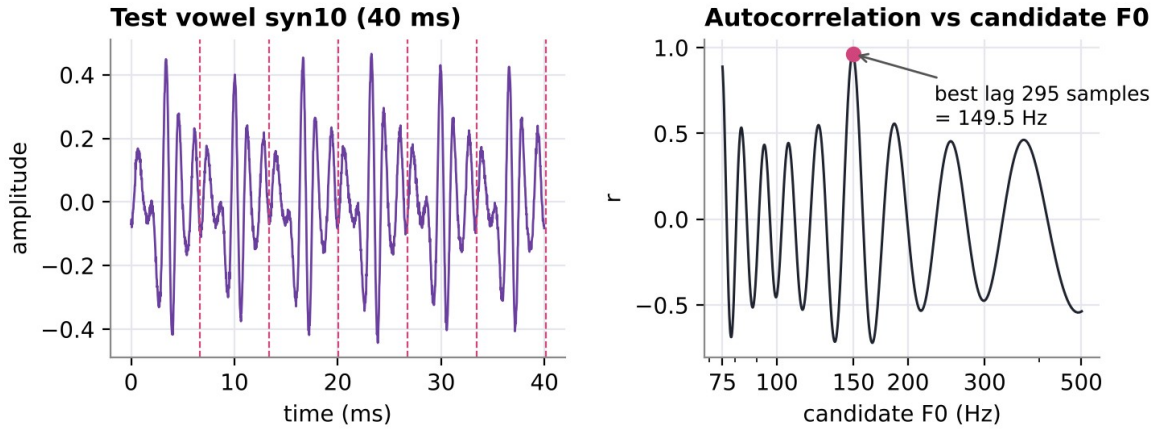


Figure 5.1: *Left: 40 ms of a test vowel with the detected period marked. Right: correlation r for each candidate pitch*

5.4 Cycle-by-Cycle Tracking

Jitter and shimmer need the length and height of *every* cycle, not just the average. Starting at time *t* with the current period estimate *T*, the analyser takes one period of signal and slides a copy forward by every shift *s* from 0.8·T to 1.2·T, keeping the shift with the highest correlation. That best shift is the length of this cycle. The search window adapts to each new period, so slow changes of pitch are followed.

Because the best shift is a whole number of samples, it can be up to half a sample (11 µs at 44.1 kHz) wrong. This is comparable to the timing differences that jitter measures, so the position of the peak is refined by fitting a parabola through the correlation values at s − 1, s and s + 1 (values *a*, *b*, *c*):

$$T = s + 0.5 \cdot (a - c) \,/\, (a - 2b + c) \tag{5.4}$$

The refined period is kept inside the 75–500 Hz range, *t* is advanced by *T*, and the largest sample value inside the cycle is stored as its peak height *A*. The correlation *b* is also added to a running sum for the HNR. The same exact time *t* is carried forward (only the window start is rounded), so the fractional part of each period is never lost — a defect that was found and fixed in the extended prototype (Section 8.3).

5.5 Jitter, Shimmer and F0

With N cycles of lengths T_1, T_2, … and peak heights A_1, A_2, …:

$$F0 = N \,/\, \Sigma\, T_i \tag{5.5}$$

$$\text{Jitter}\ (\%) = 100 \cdot \left[\, (1/(N-1))\ \Sigma\, |T_i - T_{i-1}| \,\right] / \left[\, (1/N)\ \Sigma\, T_i \,\right] \tag{5.6}$$

$$\text{Shimmer}\ (\%) = 100 \cdot \left[\, (1/(N-1))\ \Sigma\, |A_i - A_{i-1}| \,\right] / \left[\, (1/N)\ \Sigma\, A_i \,\right] \tag{5.7}$$

These are the "local" jitter and shimmer of MDVP and Praat [6], [8]. Both are relative measures, so they do not depend on the voice's pitch or the recording level.

5.6 Harmonics-to-Noise Ratio

If $\bar{r}$ is the mean of the best correlations found while tracking the cycles, then following Boersma [9]:

$$HNR\ (dB) = 10 \cdot log_{10}(\ \bar{r} \ / \ (1 - \bar{r})\ ) \tag{5.8}$$

A perfectly periodic voice has $\bar{r} \to 1$ and a very high HNR; noise lowers $\bar{r}$. For example $\bar{r}$ = 0.99 gives 20 dB. If $\bar{r}$ is below 0.5 the neighbouring cycles hardly resemble each other and the recording is rejected as noise.

5.7 Decision Rule

Table 5.1: *Reference limits used for the screening decision*

Measure	Normal if	Source
Jitter (local)	≤ 1.04 %	MDVP threshold [8]
Shimmer (local)	≤ 3.81 %	MDVP threshold [8]
HNR	≥ 20 dB	Commonly used limit [6]

Each measure outside its limit counts as one **marker** (0 to 3). If two or more markers are present the result is **FURTHER CHECK SUGGESTED (see a neurologist)**; otherwise **NO CONCERN INDICATED**. Requiring two markers makes the decision less sensitive to a single noisy measurement. The rule is fully transparent: the report prints every value and every limit, so the user can see exactly why a result was given.

5.8 Synthetic Demo Voice

Options 3 and 4 build an artificial vowel so the program can be demonstrated anywhere. Each cycle at 140 Hz is the sum of three harmonics with amplitudes 1, 0.5 and 0.25. The length of each cycle is multiplied by (1 + j·g) and its amplitude by (1 + s·g), where g is a random Gaussian number, and small Gaussian noise is added to every sample (Figure 5.2). The steady voice uses j = 0.4%, s = 2% and noise 0.005; the shaky voice uses j = 2.5%, s = 12% and noise 0.05. A fixed random seed (7) makes the demo give the same numbers every time it is run.

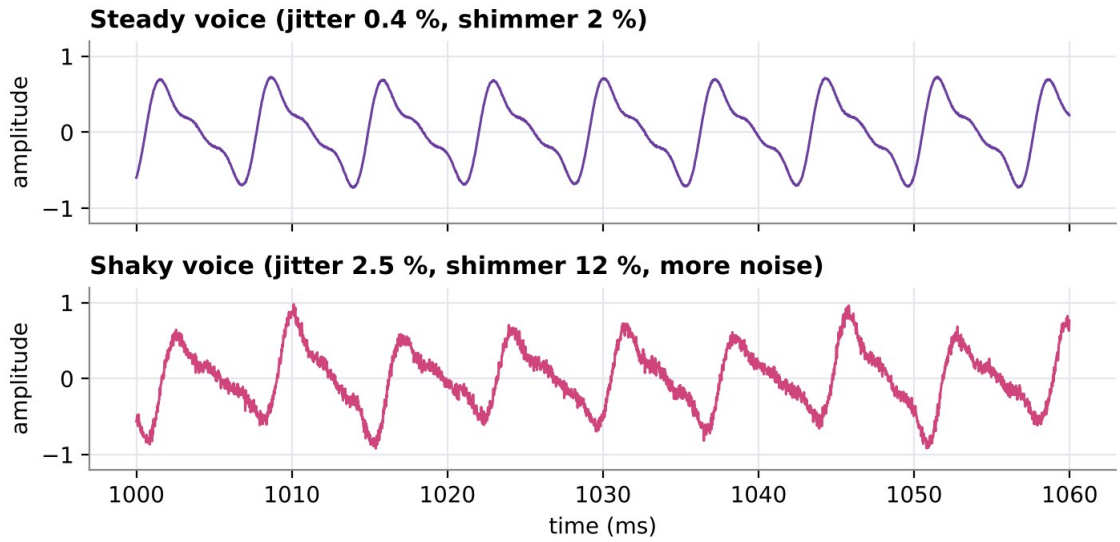


Figure 5.2: *60 ms of the steady and shaky demo voices*

5.9 Complexity Analysis

Let *n* be the number of samples analysed, *P* the period in samples and *K* the number of lags searched (about 500 at 44.1 kHz). The rough pitch search computes K correlations over about n samples: O(K·n). Cycle tracking computes 0.4·P correlations of length P for each of the n/P cycles: O(0.4·P·n). Jitter, shimmer and F0 are O(N) in the number of cycles. The overall time is therefore O((K + P)·n), and the memory is O(n) for the sample array plus O(N) for the two lists. For a 5-second recording this is a few hundred million multiplications, which a laptop completes in well under a second (Section 7.6).

CHAPTER 6

IMPLEMENTATION

6.1 Development Environment

The code was written in Java 17 and tested on JDK 17 and 21; it uses no feature newer than Java 11 (`readAllBytes`, multi-catch, try-with-resources, lambdas). It was developed in VS Code and IntelliJ IDEA, version-controlled with Git and hosted on GitHub (github.com/vudigapranav/neurovox, folder `exam-version/`).

6.2 Package audio — class VoiceSource

`VoiceSource` is the abstract base class. It declares the constant `RATE` as `public static final`, holds the samples in a `protected double[]` so that subclasses can fill it, and provides the shared helper `toSamples()`. It overrides `Object.toString()` to describe the recording. The abstract methods force every subclass to say how it loads samples and what it is called:

```
public abstract class VoiceSource {
    public static final float RATE = 44100f;
    protected double[] samples;
    protected float rate;
    protected VoiceSource(float rate) { this.rate = rate; }
    public abstract void load() throws VoiceException;
    public abstract String getName();
    ...
}
```

The three subclasses are **static nested classes** of `VoiceSource`. A Java file may contain only one public top-level class, so nesting them is how all four public types can live in one file and still be used from outside the package. Each constructor calls `super(RATE)`, and `Microphone()` and `Demo()` show constructor overloading with `this(...)`:

```
public static class Microphone extends VoiceSource {
    private final int seconds;
    public Microphone() { this(5); }
    public Microphone(int seconds) { super(RATE); this.seconds = seconds; }
    public void load() throws VoiceException {
        try {
            AudioFormat fmt = new AudioFormat(RATE, 16, 1, true, false);
            TargetDataLine mic = AudioSystem.getTargetDataLine(fmt);
            ...
        } catch (LineUnavailableException | IOException | IllegalArgumentException
e) {
            throw new VoiceException("Microphone not available: " + e.getMessage());
        }
    }
}
```

`Microphone.load()` opens a `TargetDataLine` for 44.1 kHz, 16-bit, signed, little-endian mono audio, reads exactly seconds × 88,200 bytes, saves them with `AudioSystem.write` and converts them to samples. `WavFile.load()` uses try-with-resources so the `AudioInputStream` is always closed, and checks that the file is 16-bit little-endian PCM. Library exceptions are caught and *re-thrown* as `VoiceException`, so callers deal with only one exception type.

6.3 Package analysis — classes PitchTracker and VoiceReport

The analysis package has two short classes with one job each. **PitchTracker** does all the signal processing in its constructor: the length and loudness checks, the rough pitch search and the cycle-by-cycle tracking of Chapter 5. If anything is wrong it throws `VoiceException`, so a `PitchTracker` object only ever exists for a usable recording. Its results are stored in two `ArrayList<Double>` (cycle lengths and peak heights) and one `double` (the mean similarity), declared without an access modifier so that `VoiceReport`, in the same package, can read them directly. The heart of the class is the tracking loop:

```
while (t + 2.5 * period < x.length) {
    int pos = (int) Math.round(t), n = (int) period, s = (int) (0.8 * period);
    double b = corr(x, pos, pos + s, n);
    for (int k = s + 1; k <= 1.2 * period; k++) {
        double c = corr(x, pos, pos + k, n);
        if (c > b) { b = c; s = k; }
    }
    double a = corr(x, pos, pos + s - 1, n), c = corr(x, pos, pos + s + 1, n), d = a
- 2 * b + c;
    period = d < 0 ? s + 0.5 * (a - c) / d : s;
    period = Math.max(minLag, Math.min(maxLag, period));
    ...  // store period, peak height and b
    t += period;
}
```

VoiceReport turns those lists into the four measures. Because jitter and shimmer use the same formula on different lists (equations 5.6 and 5.7), two small helper methods, `mean(List<Double>)` and `change(List<Double>)`, cover both, which keeps the class short. Its `run()` method shows try, two catch blocks and finally:

```
public void run() {
    try {
        PitchTracker p = new PitchTracker(voice);
        features.put(F0, 1 / mean(p.periods));
        features.put(JITTER, 100 * change(p.periods) / mean(p.periods));
        features.put(SHIMMER, 100 * change(p.peaks) / mean(p.peaks));
        features.put(HNR, 10 * Math.log10(p.similarity / (1 - p.similarity)));
    } catch (VoiceException e) {
        error = e.getMessage();
    } catch (RuntimeException e) {
        error = "Analysis failed: " + e;
```

```
    } finally {
        finish();                    // synchronized: done = true
    }
}
```

The measures are kept in a `LinkedHashMap<String, Double>`, which keeps insertion order so the report always lists F0, jitter, shimmer and HNR in the same order. `compareTo()` orders two reports by jitter, so `Collections.sort(session)` lists the steadiest voice first; `toString()` gives a one-line summary for the comparison screen and `toCsv()` a comma-separated line for the history file.

6.4 Class Main

`main()` reads the patient's name with a `Scanner` and runs a `do-while` menu loop with a `switch` statement. The variable `voice` has the superclass type `VoiceSource`, and is assigned a `Microphone`, `WavFile` or `Demo` object depending on the choice. `screen()` then loads the voice, starts the analysis thread, waits for it and prints the report:

```
VoiceReport a = new VoiceReport(voice);
Thread worker = new Thread(a, "analyzer");
worker.start();
System.out.print("Analysing");
while (!a.isDone()) {
    System.out.print(".");
    Thread.sleep(50);
}
worker.join();
System.out.println("\n" + a.report());
```

History is written with a `BufferedWriter` wrapped round a `FileWriter` opened in append mode, and read back with a `BufferedReader`; each line is split with a `StringTokenizer` on commas. `compare()` sorts the session list, walks it with an explicit `Iterator`, and counts flagged voices with a stream and a lambda expression. The generic method `<T> printAll(String, List<T>)` prints any list.

6.5 Mapping to the Java Programming Syllabus (22ITC02N)

Table 6.1 shows where each syllabus topic [15] appears in the code, so that the project can serve as a revision aid.

Table 6.1: *Syllabus topics and where they are used*

Unit	Topic	Where in the code
I	Classes, objects, fields, methods, access modifiers	All four files (private / protected / default / public)
I	Constructors and constructor overloading	Microphone(), Microphone(int), Demo(), Demo(boolean)
I	static and final	RATE, the limits, final class WavFile, final fields

Unit	Topic	Where in the code
I	Arrays, Strings, StringTokenizer, Scanner	double[] samples; String.format; showHistory(); main()
I	Flow control	switch, do-while, while, for, for-each, ternary operator
II	Inheritance and super	Microphone, WavFile, Demo extend VoiceSource; super(RATE)
II	Preventing inheritance	public static final class WavFile
II	Object class, method overriding	toString() in VoiceSource and VoiceReport
II	Dynamic method dispatch	voice.load() on a VoiceSource reference
II	Abstract classes and methods	VoiceSource, load(), getName()
II	Interfaces	VoiceReport implements Runnable, Comparable
II	Inner classes	Four static nested classes in VoiceSource
II	Packages, import, CLASSPATH	package audio; package analysis; import audio.VoiceSource.*
III	User-defined exception, throw, throws	VoiceException; PitchTracker(…) throws VoiceException
III	try, multiple catch, multi-catch, finally	run(), screen(), Microphone.load(), showHistory()
III	Threads, life cycle, synchronization	new Thread, start(), sleep(), join(); synchronized isDone()
III	Generics	List<Double>, Map<String, Double>, Comparable<VoiceReport>, mean(List<Double>), <T> printAll()
IV	ArrayList, LinkedHashMap, Iterator	periods, peaks, features, session; compare()
IV	Comparable interface	compareTo() + Collections.sort()
IV	Java 8: lambda expressions, streams	session.stream().filter(v -> v.needsFurtherCheck())
V	FileWriter, BufferedWriter, FileReader, BufferedReader	saveHistory(), showHistory()
V	Byte streams	AudioInputStream, ByteArrayInputStream, readAllBytes()

6.6 Compiling and Running

From the folder that contains `Main.java`:

```
javac Main.java audio/*.java analysis/*.java
java Main
```

`javac` compiles the four files and places `VoiceSource.class` (and its nested classes, such as `VoiceSource$Demo.class`) in `audio/` and `PitchTracker.class` and `VoiceReport.class` in `analysis/`, matching the package declarations. `java Main` runs the program with the current folder as the CLASSPATH. On Windows the paths are written `audio\*.java`; the file `run.bat` does both steps.

CHAPTER 7

TESTING AND RESULTS

7.1 Testing Strategy

Testing was done at three levels: (1) **functional tests** of every menu option and error path, (2) **known-answer tests** with synthetic voices whose jitter and shimmer were chosen in advance, and (3) an **accuracy study** on 40 realistic test vowels with exactly known cycle lengths, generated by the validation script of the extended prototype.

7.2 Test Cases

Table 7.1: *Functional test cases*

ID	Input	Expected	Actual	Status
TC01	Option 3 (steady demo)	0 markers, NO CONCERN	Jitter 0.30 %, shimmer 2.35 %, HNR 35.1 dB, 0 markers	Pass
TC02	Option 4 (shaky demo)	≥ 2 markers, FURTHER CHECK	Jitter 2.21 %, shimmer 13.57 %, HNR 16.6 dB, 3 markers	Pass
TC03	Option 2, test vowel syn10.wav	F0 ≈ 149.5 Hz	F0 149.3 Hz, NO CONCERN	Pass
TC04	Option 2, stereo WAV	First channel analysed	Same values as mono file	Pass
TC05	Option 2, 4 s of silence	Rejected: too quiet	"Too quiet. Say 'aah' louder."	Pass
TC06	Option 2, white noise only	Rejected: no voice	"No clear voice found (only noise)…"	Pass
TC07	Option 2, 0.1 s clip	Rejected: too short	"Too short. Record at least 2 seconds."	Pass
TC08	Option 2, 32-bit float WAV	Rejected: format	"Please use a 16-bit PCM WAV file"	Pass
TC09	Option 2, text file named .wav	Rejected: unreadable	"Cannot read …: File of unsupported format"	Pass
TC10	Option 2, missing file	Rejected: not found	"Cannot read … (No such file or directory)"	Pass
TC11	Option 1 on a machine without microphone	Clear message, no crash	"Microphone not available: …"	Pass
TC12	Menu input 9, −3, "abc"	Invalid choice, menu again	"Invalid choice."	Pass
TC13	Option 5 after three analyses	Sorted by jitter, count of flags	Steady, syn10, shaky; 1 of 3 flagged	Pass
TC14	Option 6	All saved rows shown	Rows parsed with StringTokenizer	Pass
TC15	Option 5 / 6 before any analysis	Friendly message	"Nothing analysed yet." / "No history yet."	Pass
TC16	Empty patient name	Default name used	Saved as "Patient"	Pass

7.3 Output Screens

Figures 7.1 to 7.5 show the console during a typical session.

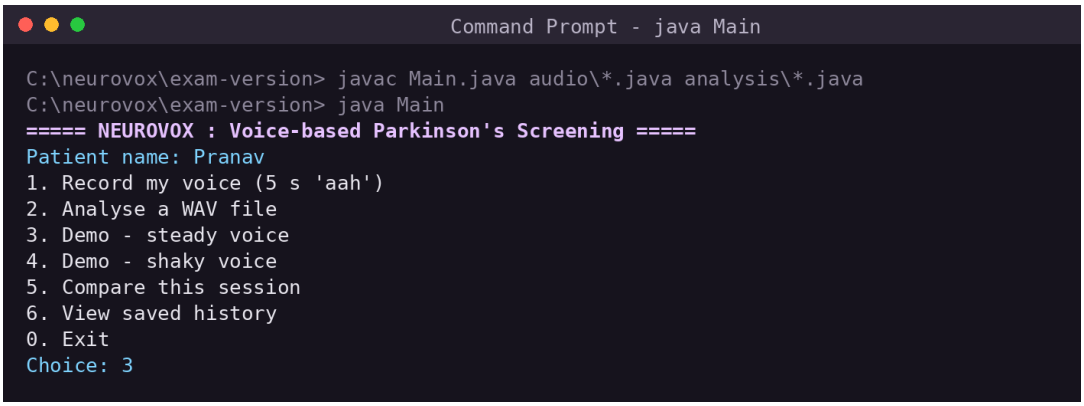


Figure 7.1: *Compiling and starting the program; the main menu*



Figure 7.2: *Result for the steady demo voice: no marker outside the limits*

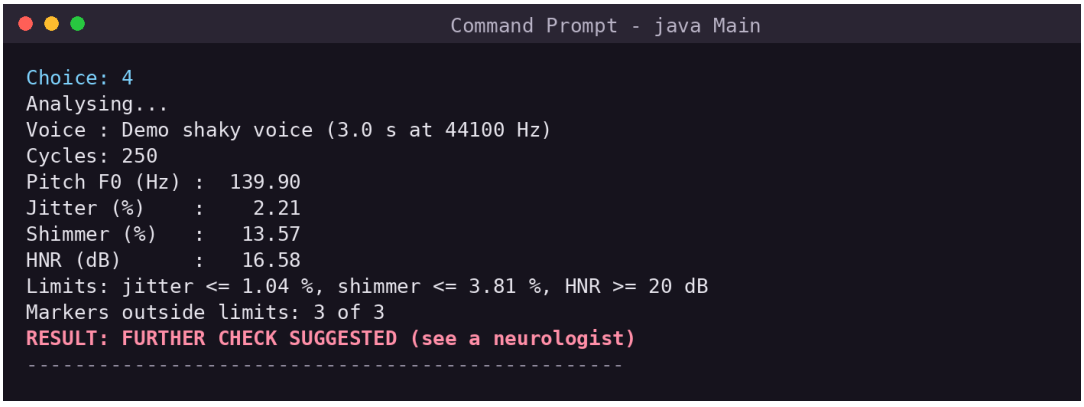


Figure 7.3: *Result for the shaky demo voice: all three markers outside the limits*

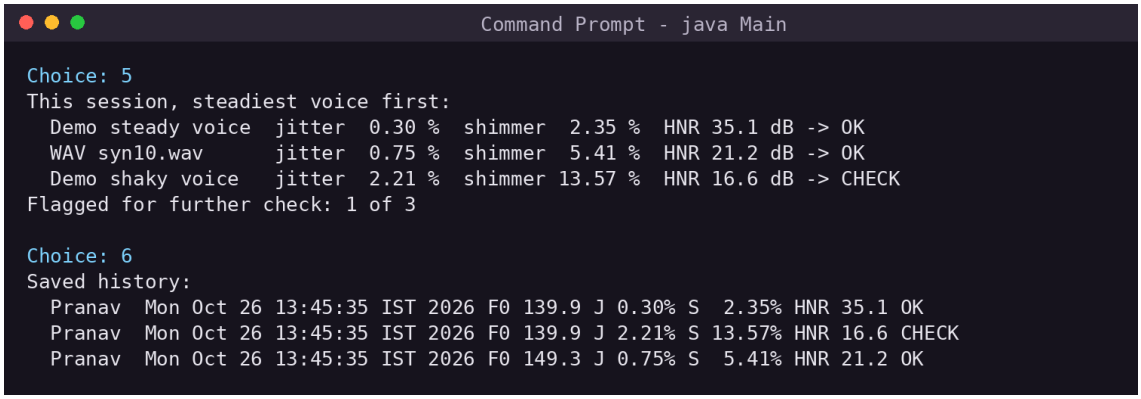


Figure 7.4: *Session comparison (sorted by jitter) and the saved history file*

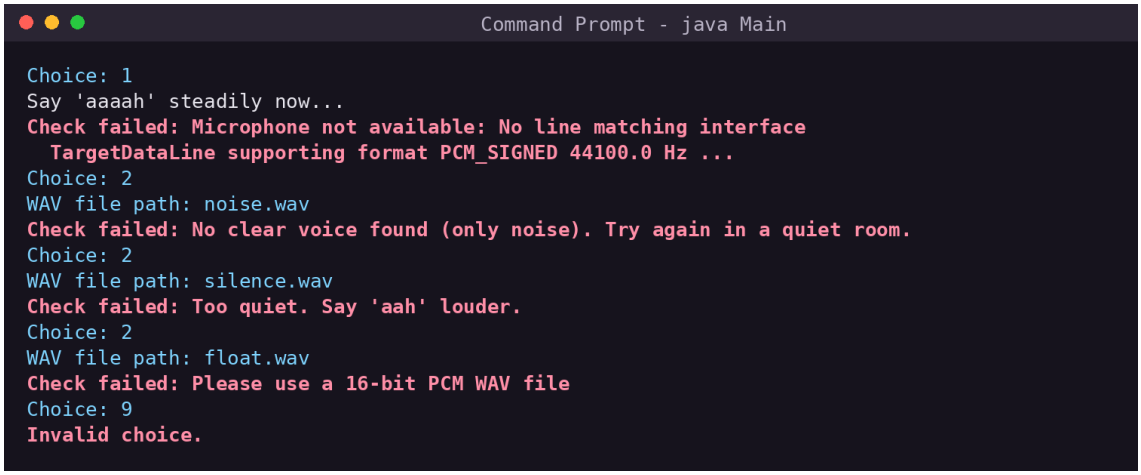


Figure 7.5: *Error handling: no microphone, noise, silence, wrong format and invalid menu input*

7.4 Accuracy on Test Vowels with Known Values

Real patient recordings with known jitter do not exist — jitter is defined by the measuring program. To check correctness, the 40 synthetic sustained /a/ vowels of the extended prototype's validation suite were used. They are not simple sine waves: each cycle is a glottal pulse (Rosenberg shape [16]) passed through three formant resonators at 730, 1090 and 2440 Hz, as in a real /a/, with F0 between 94 and 245 Hz, injected jitter of 0.2–2.5%, shimmer of 1–10% and added noise at 12–35 dB signal-to-noise ratio. The script records the exact length and amplitude of every cycle, so the true jitter and shimmer are known. All 40 files were analysed by the console version through menu option 2.

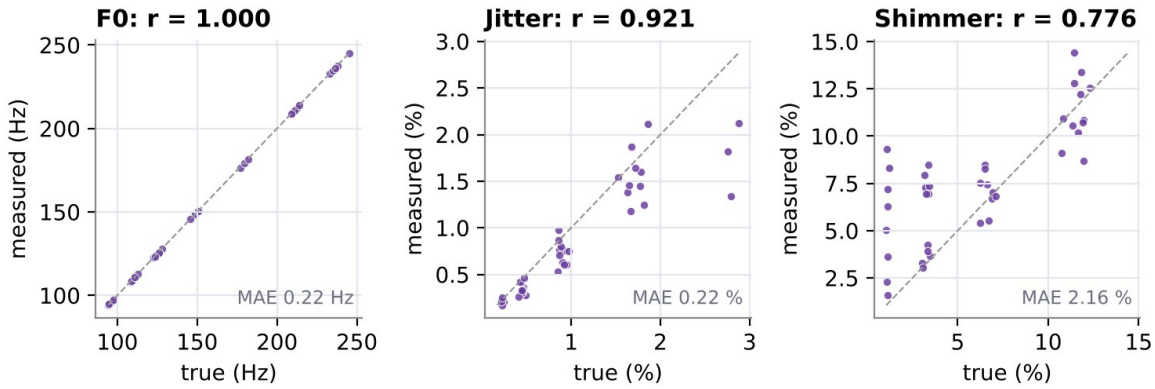


Figure 7.6: *Console version against the known true values for 40 test vowels (dashed line = perfect agreement)*

Table 7.2: *Accuracy of the console version on 40 test vowels*

Measure	Mean absolute error	Mean bias	Correlation r
F0	0.22 Hz	-0.22 Hz	1.000
Jitter	0.22 %-points	-0.19 %-points	0.921
Shimmer	2.16 %-points	+1.56 %-points	0.776
HNR vs added SNR	—	—	0.85

F0 is essentially exact (r = 1.000, error 0.22 Hz). **Jitter** follows the truth closely (r = 0.92) with a small tendency to read low, especially for the three files with the highest jitter (about 2.8%). This is expected: the correlation window is one full period and overlaps its neighbours, which smooths the largest perturbations. Praat shows the same behaviour — in the extended prototype, Praat and the full Java implementation both under-read jitter by about 0.14 percentage points against the same truth (Section 8.3).

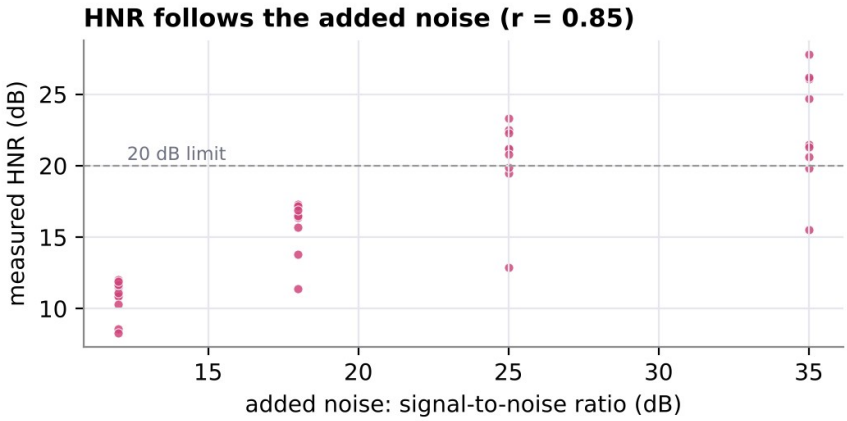


Figure 7.7: *Measured HNR against the amount of added noise*

HNR decreases as more noise is added (r = 0.85 with the signal-to-noise ratio; Figure 7.7). **Shimmer** is the weakest measure (r = 0.78) and reads high on noisy files, because noise is

added on top of every peak. Table 7.3 shows that this over-reading disappears in quiet recordings, which is why the program and the user manual ask for a quiet room.

Table 7.3: *Shimmer over-reading against the noise level*

Added noise (SNR)	Files	Mean shimmer error
12 dB (very noisy)	13	+3.29 %-points
18 dB	9	+1.84 %-points
25 dB	9	+0.52 %-points
35 dB (quiet)	9	−0.17 %-points

7.5 Agreement of the Final Decision

For each test vowel, the *reference* decision was computed by applying the same rule to the true jitter, the true shimmer and the added signal-to-noise ratio (as a stand-in for HNR). The console version agreed with the reference on **33 of 40 files (82.5%)**. It never missed a file that the reference flagged (19 of 19), and gave 7 extra flags. All 7 extra flags were noisy files where over-read shimmer pushed the count of markers to two; the jitter marker agreed on all 40 files and the HNR marker on 35. In a screening setting, an extra flag (a needless visit) is a smaller harm than a miss, but the result again shows that recording conditions matter.

Table 7.4: *Decision agreement on 40 test vowels*

	Reference: flag	Reference: no flag
Console: FURTHER CHECK	19	7
Console: NO CONCERN	0	14

7.6 Performance

All 40 files (3.5–4.5 s each) were analysed in a single run in 10.8 seconds including Java start-up — about 0.27 s per file — on a cloud Linux machine. A 3-second demo voice is analysed in under 0.2 s, which is why only a few progress dots appear. Memory use is dominated by the sample array: 8 bytes × 220,500 samples ≈ 1.8 MB for a 5-second recording.

CHAPTER 8

EXTENDED WEB PROTOTYPE AND RESEARCH FINDINGS

8.1 Overview

Before the exam version, the team built a full local web application, also called Neurovox (about 10,000 lines of Java and JavaScript). A Java 17 process uses the JDK's built-in `HttpServer` (deliberately not Spring Boot, so that every route is visible in code), stores data in SQLite, and does all signal processing in Java. The browser guides the user through four voice tasks — a sustained vowel, the syllables "pa-ta-ka", a reading passage and free speech — while a particle orb reacts to the microphone (Figure 8.1). Java checks each recording's quality before storing it, then computes the 16 measures of the UCI dataset (jitter, shimmer and HNR variants, F0 statistics) plus MFCCs and speech-timing measures, and shows a report and a history of trends (Figures 8.2 and 8.3). The prototype has 174 Java unit tests and 31 frontend tests.

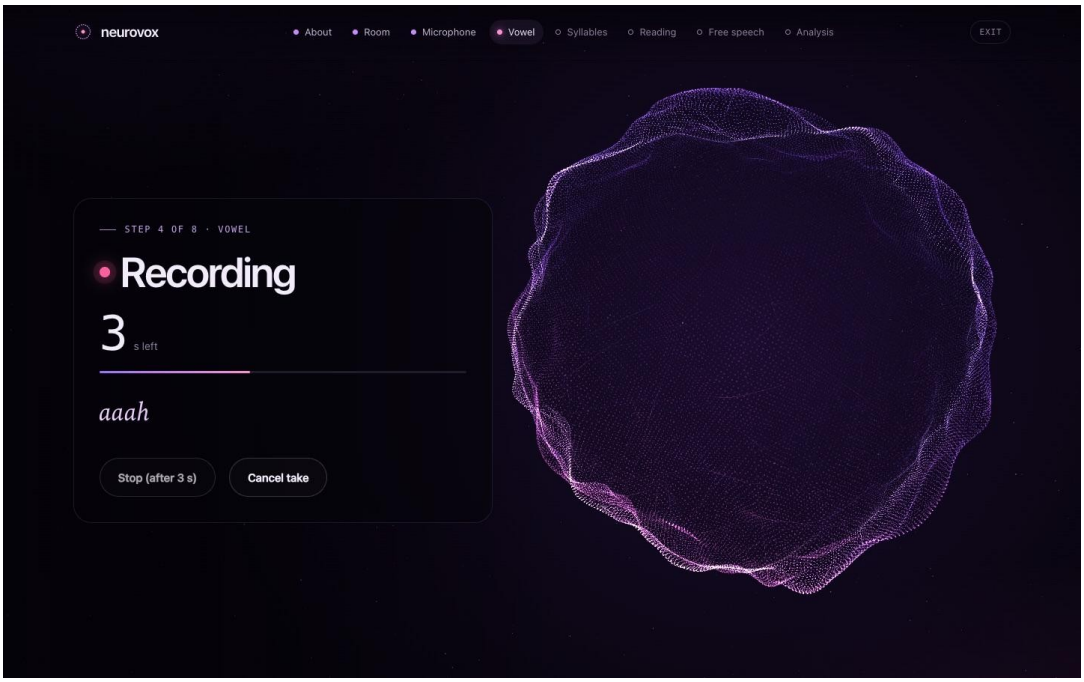


Figure 8.1: *Web prototype: recording the sustained vowel; the orb reacts to the voice*

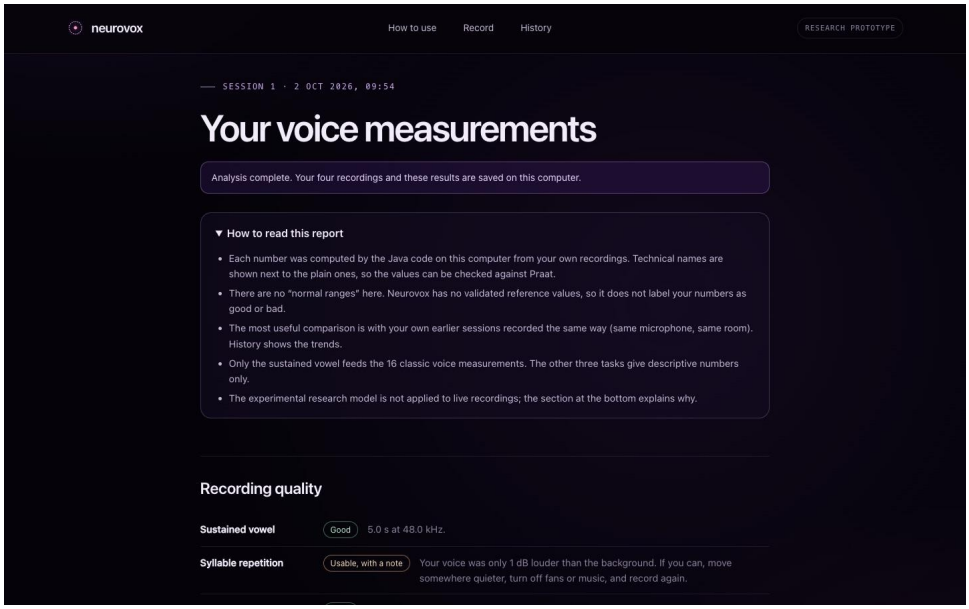


Figure 8.2: *Web prototype: top of the measurement report, with recording quality*

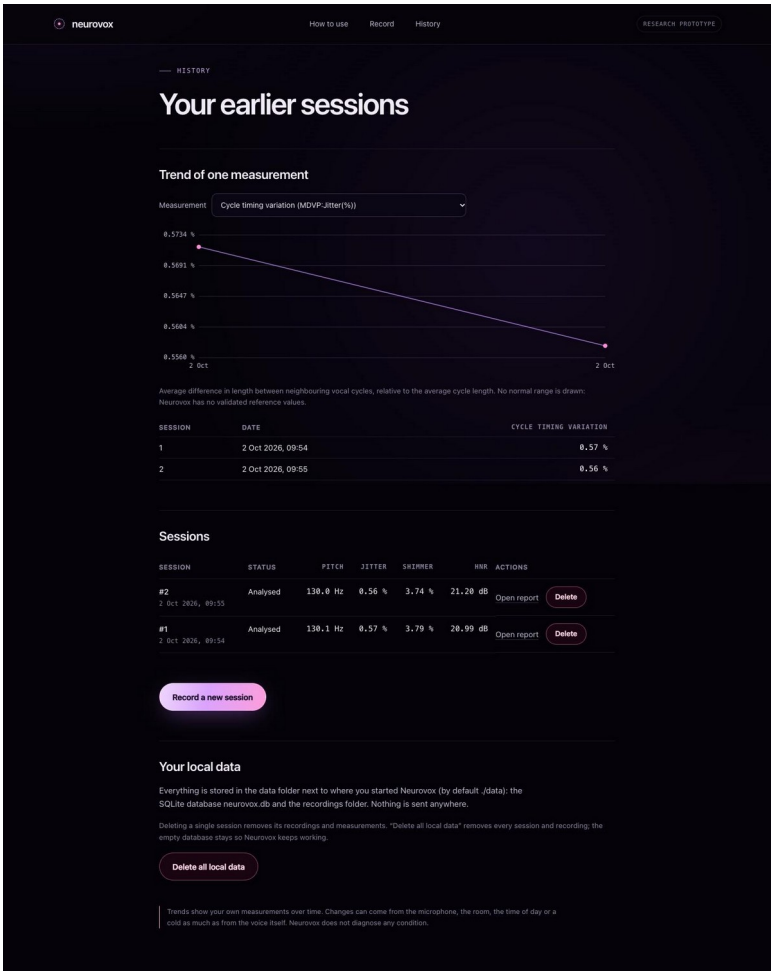


Figure 8.3: *Web prototype: history page with the trend of one measurement*

8.2 Java Versus Praat

The full implementation was compared with Praat 6.1.38, the reference program in speech science, on the same 40 synthetic vowels, the same analysis region and the same settings.

All 16 measurements agreed within 0.15% relative error with a correlation of 1.000 (Table 8.1, Figure 8.4). Two defects were found and fixed during this study: pulse positions had been rounded to whole samples, losing the fractional part of each period, and cycle timing had been measured on a 16 kHz copy of the signal instead of at the original rate. The console version avoids both by design (Section 5.4).

Table 8.1: *Full Java implementation against Praat on 40 vowels (selected measures)*

Measure	Java mean	Praat mean	Relative MAE	r
F0 (Hz)	151.669	151.669	0.00 %	1.000
Jitter (local)	0.0089	0.0089	0.12 %	1.000
RAP	0.0051	0.0050	0.15 %	1.000
Shimmer (local)	0.0678	0.0679	0.11 %	1.000
NHR	0.0725	0.0725	0.12 %	1.000
HNR (dB)	14.549	14.543	0.07 %	1.000

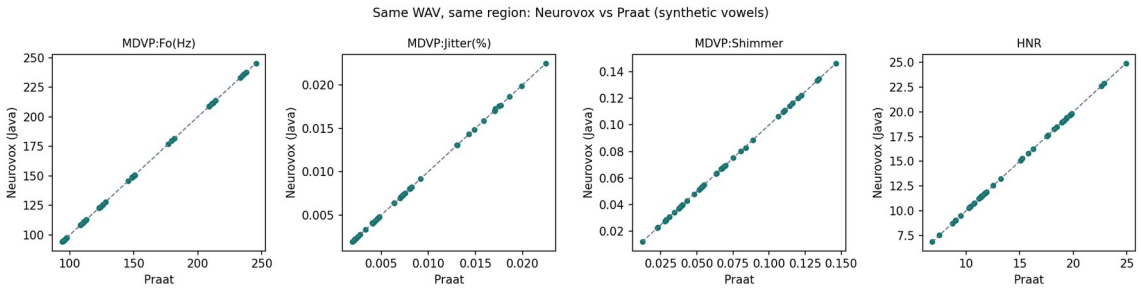


Figure 8.4: *Full Java implementation against Praat for each measurement*

8.3 Autocorrelation Versus YIN

Both pitch detectors were implemented and run on the 40 vowels under three conditions: as generated, with the fundamental removed by a 300 Hz high-pass filter (the classic trap for octave errors), and with noise at 6 dB SNR (Table 8.2, Figure 8.5). Autocorrelation was more accurate in every condition; YIN was about 16 times faster. The choice also matters downstream: on individual noisy files, jitter computed with YIN-guided pulses differed from autocorrelation-guided jitter by up to 224%, because YIN's octave jumps break the cycle tracking.

Table 8.2: *Pitch detector comparison (40 vowels)*

Condition	Detector	Mean error	Octave errors	ms per s audio
Clean	Autocorrelation	0.77 Hz	0.00 %	48.7
Clean	YIN	3.25 Hz	1.80 %	3.0
High-pass 300 Hz	Autocorrelation	1.06 Hz	0.29 %	46.2
High-pass 300 Hz	YIN	3.74 Hz	1.96 %	3.0
Noise 6 dB	Autocorrelation	0.79 Hz	0.00 %	30.9

Condition	Detector	Mean error	Octave errors	ms per s audio
Noise 6 dB	YIN	15.27 Hz	10.61 %	3.5

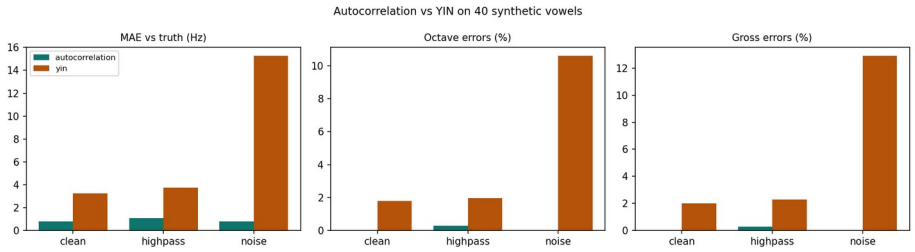


Figure 8.5: *Pitch error and speed of the two detectors*

8.4 The Research Model and the Go/No-Go Decision

A logistic-regression model was trained in Python (scikit-learn) on the 16 measures of the UCI dataset — 195 recordings from 32 subject codes — and exported to Java, where inference matched scikit-learn to within 10^{-9}. The crucial question was how to test it. Table 8.3 shows that the leaky random split, which lets the same person appear in training and testing, gives 80.5% accuracy, but grouping the folds by person reduces it to 69.2%, *below* the 75.4% baseline of always answering "Parkinson's". Specificity — correctly clearing healthy people — was only 33%.

Table 8.3: *Research model evaluation (positive class = Parkinson's)*

Evaluation	Accuracy	Balanced acc.	Sensitivity	Specificity	ROC-AUC
Majority-class baseline	75.4 %	50.0 %	100 %	0 %	—
Random split (leaky)	80.5 %	69.5 %	91.2 %	47.9 %	0.820
Grouped by person, 5-fold	69.2 %	57.1 %	81.0 %	33.3 %	0.622
Leave one person out	73.8 %	61.6 %	85.7 %	37.5 %	0.660

In addition, the dataset contains measurements only — no audio and no extraction settings — so there is no way to confirm that values computed by our code (or by Praat) are on the same scale as the MDVP values the model was trained on. The team therefore recorded a **no-go** decision: the model is reported as an offline research result and is never applied to live recordings.

8.5 Consequences for the Console Version

- The console version uses **transparent reference limits** instead of the trained model, prints the limits with every result, and gives a **binary** indication, since no probability from this data would be trustworthy.
- It uses **autocorrelation** (not YIN), and tracks cycles at the **original sample rate** with **fractional** cycle lengths.

CHAPTER 9

LIMITATIONS, ETHICS AND FUTURE SCOPE

9.1 Limitations

- **No human validation.** Accuracy was measured on synthetic vowels with known answers. Real voices are less regular, and the program has not been tested on recordings from people with and without Parkinson's disease.
- **Thresholds from another program.** The 1.04% and 3.81% limits were defined for MDVP's own algorithms. Our jitter and shimmer follow the same definitions, but small differences in method can shift values near the limits.
- **Shimmer is sensitive to noise** (Table 7.3), so a noisy room can produce a false flag.
- **One task only.** The console version analyses a sustained vowel; the speech tasks of the web prototype (syllable rate, pauses, monopitch in reading) are not included.
- **Not specific to Parkinson's disease.** A cold, smoking, vocal fatigue, ageing and many voice disorders raise jitter and shimmer too.

9.2 Ethical Considerations

A health-related tool must not cause harm through false confidence. Neurovox never uses the word "diagnosis", always prints the limits behind its result, and phrases a positive result as a suggestion to see a neurologist. All recordings and history stay on the user's computer; nothing is uploaded. Voice is personal data, so the history file contains only the name typed by the user and the measurements, not the audio. Any future clinical use would require ethical approval, informed consent and regulatory review as a medical device.

9.3 Future Scope

- Record 20 or more human vowels with consent and compare the console version with Praat on them.
- Add the syllable-repetition task ("pa-ta-ka") to measure articulation rate, the vocal equivalent of the finger-tapping test.
- Replace the fixed limits with each user's own baseline, so that changes over months are tracked rather than single values.
- Add a simple JavaFX or web front end to the console core, reusing the same two packages.
- Collect a properly consented dataset with recordings (not only features) so that a classifier can be validated by person.

CHAPTER 10

CONCLUSION

Neurovox shows that a complete voice-screening pipeline — microphone capture, pitch detection, cycle-by-cycle tracking, jitter, shimmer, harmonics-to-noise ratio and a transparent decision rule — can be written in about 395 lines of standard Java, organised into a `Main` class and two packages in four short files, and small enough to be written by hand in an examination. The design uses the object-oriented features taught in the Java Programming course in a natural way: an abstract class with overridden methods for the different voice sources, a user-defined checked exception for every recoverable problem, a worker thread with synchronised state, generic collections for the measurements, and buffered file I/O for the history.

Testing on 40 realistic synthetic vowels with known values showed that the console version measures pitch almost exactly (0.22 Hz error) and jitter closely ($r = 0.92$), and that its final decision agreed with the reference decision on 33 of 40 files with no missed cases. The earlier web prototype contributed two lessons that shaped this version: a careful implementation can match Praat to within 0.15%, but a classifier trained on the public dataset does not generalise to new people when tested honestly. For that reason Neurovox remains what it set out to be — a screening aid that measures, explains and suggests a doctor's visit, and never a diagnosis.

REFERENCES

[1] World Health Organization, "Parkinson disease," Fact sheet, 2023. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/parkinson-disease

[2] R. B. Postuma *et al.*, "MDS clinical diagnostic criteria for Parkinson's disease," *Movement Disorders*, vol. 30, no. 12, pp. 1591–1601, 2015.

[3] J. A. Logemann, H. B. Fisher, B. Boshes and E. R. Blonsky, "Frequency and cooccurrence of vocal tract dysfunctions in the speech of a large sample of Parkinson patients," *Journal of Speech and Hearing Disorders*, vol. 43, no. 1, pp. 47–57, 1978.

[4] J. Rusz, R. Cmejla, H. Ruzickova and E. Ruzicka, "Quantitative acoustic measurements for characterization of speech and voice disorders in early untreated Parkinson's disease," *Journal of the Acoustical Society of America*, vol. 129, no. 1, pp. 350–367, 2011.

[5] M. A. Little, P. E. McSharry, E. J. Hunter, J. Spielman and L. O. Ramig, "Suitability of dysphonia measurements for telemonitoring of Parkinson's disease," *IEEE Transactions on Biomedical Engineering*, vol. 56, no. 4, pp. 1015–1022, 2009.

[6] J. P. Teixeira, C. Oliveira and C. Lopes, "Vocal acoustic analysis – jitter, shimmer and HNR parameters," *Procedia Technology*, vol. 9, pp. 1112–1122, 2013.

[7] P. Boersma and D. Weenink, *Praat: doing phonetics by computer* [Computer program], version 6.1.38. [Online]. Available: https://www.praat.org

[8] Kay Elemetrics Corp., *Multi-Dimensional Voice Program (MDVP), Model 5105: Software Instruction Manual*. Lincoln Park, NJ, USA, 1993.

[9] P. Boersma, "Accurate short-term analysis of the fundamental frequency and the harmonics-to-noise ratio of a sampled sound," *Proceedings of the Institute of Phonetic Sciences, University of Amsterdam*, vol. 17, pp. 97–110, 1993.

[10] A. de Cheveigné and H. Kawahara, "YIN, a fundamental frequency estimator for speech and music," *Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002.

[11] M. A. Little, P. E. McSharry, S. J. Roberts, D. A. E. Costello and I. M. Moroz, "Exploiting nonlinear recurrence and fractal scaling properties for voice disorder detection," *BioMedical Engineering OnLine*, vol. 6, art. 23, 2007.

[12] M. Little, "Parkinsons," UCI Machine Learning Repository, 2008. [Online]. Available: https://archive.ics.uci.edu/dataset/174/parkinsons

[13] A. Tsanas, M. A. Little, P. E. McSharry and L. O. Ramig, "Accurate telemonitoring of Parkinson's disease progression by noninvasive speech tests," *IEEE Transactions on Biomedical Engineering*, vol. 57, no. 4, pp. 884–893, 2010.

[14] C. O. Sakar *et al.*, "A comparative analysis of speech signal processing algorithms for Parkinson's disease classification and the use of the tunable Q-factor wavelet transform," *Applied Soft Computing*, vol. 74, pp. 255–263, 2019.

[15] Chaitanya Bharathi Institute of Technology (A), *Scheme of Instruction and Syllabi (R-22A), Computer Engineering and Technology*, course 22ITC02N Java Programming, 2026.

[16] A. E. Rosenberg, "Effect of glottal pulse shape on the quality of natural vowels," *Journal of the Acoustical Society of America*, vol. 49, no. 2B, pp. 583–590, 1971.

[17] H. Schildt, *Java: The Complete Reference*, 12th ed. New York: McGraw-Hill, 2020.

[18] E. Balagurusamy, *Programming with Java*, 6th ed. New Delhi: McGraw-Hill Education, 2019.

[19] Oracle, "Java Sound API — Package javax.sound.sampled," Java SE 17 API Specification. [Online]. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/javax/sound/sampled/package-summary.html

[20] Y. Jadoul, B. Thompson and B. de Boer, "Introducing Parselmouth: a Python interface to Praat," *Journal of Phonetics*, vol. 71, pp. 1–15, 2018.

APPENDIX A: SOURCE CODE

The complete source of the console version, exactly as in the repository folder `exam-version/`. Line numbers are for reference only and are not part of the code.

A.1 Main.java (default package)

```
 1  import analysis.VoiceReport;
 2  import audio.VoiceSource;
 3  import audio.VoiceSource.*;
 4
 5  import java.io.*;
 6  import java.util.*;
 7
 8  // NEUROVOX - voice-based Parkinson's screening (console version).
 9  // It does not diagnose: it only flags voices that should be checked by a doctor.
10  public class Main {
11      static final String HISTORY = "history.csv";
12      static List<VoiceReport> session = new ArrayList<>();
13
14      public static void main(String[] args) {
15          Scanner in = new Scanner(System.in);
16          System.out.println("===== NEUROVOX : Voice-based Parkinson's Screening =====");
17          System.out.print("Patient name: ");
18          String name = in.nextLine().trim();
19          if (name.isEmpty()) name = "Patient";
20
21          int choice;
22          do {
23              System.out.println("\n1. Record my voice (5 s 'aah')\n2. Analyse a WAV file\
n3. Demo - steady voice"
24                      + "\n4. Demo - shaky voice\n5. Compare this session\n6. View saved
history\n0. Exit");
25              System.out.print("Choice: ");
26              choice = in.hasNextInt() ? in.nextInt() : -1;
27              in.nextLine();
28
29              VoiceSource voice = null;            // superclass reference
30              switch (choice) {
31                  case 1: voice = new Microphone(); break;
32                  case 2:
33                      System.out.print("WAV file path: ");
34                      voice = new WavFile(in.nextLine().trim());
35                      break;
36                  case 3: voice = new Demo(); break;
37                  case 4: voice = new Demo(true); break;
38                  case 5: compare(); break;
39                  case 6: showHistory(); break;
40                  case 0: System.out.println("Goodbye."); break;
41                  default: System.out.println("Invalid choice.");
42              }
43              if (voice != null) screen(name, voice);
44          } while (choice != 0);
45          in.close();
46      }
```

```
 47
 48      static void screen(String name, VoiceSource voice) {
 49          try {
 50              if (voice instanceof Microphone) System.out.println("Say 'aaaah' steadily
now...");
 51              voice.load();                       // dynamic method dispatch
 52
 53              VoiceReport a = new VoiceReport(voice);
 54              Thread worker = new Thread(a, "analyzer");
 55              worker.start();
 56              System.out.print("Analysing");
 57              while (!a.isDone()) {
 58                  System.out.print(".");
 59                  Thread.sleep(50);
 60              }
 61              worker.join();
 62
 63              System.out.println("\n" + a.report());
 64              session.add(a);
 65              saveHistory(name, a);
 66          } catch (VoiceException e) {
 67              System.out.println("\nCheck failed: " + e.getMessage());
 68          } catch (InterruptedException | IOException e) {
 69              System.out.println("\nError: " + e.getMessage());
 70          } finally {
 71              System.out.println("-------------------------------------------------");
 72          }
 73      }
 74
 75      static void compare() {
 76          if (session.isEmpty()) {
 77              System.out.println("Nothing analysed yet.");
 78              return;
 79          }
 80          Collections.sort(session);              // uses compareTo (lowest jitter first)
 81          System.out.println("This session, steadiest voice first:");
 82          Iterator<VoiceReport> it = session.iterator();
 83          while (it.hasNext()) System.out.println("  " + it.next());
 84          long flagged = session.stream().filter(v -> v.needsFurtherCheck()).count();   //
lambda
 85          System.out.println("Flagged for further check: " + flagged + " of " +
session.size());
 86      }
 87
 88      static void saveHistory(String name, VoiceReport a) throws IOException {
 89          try (BufferedWriter w = new BufferedWriter(new FileWriter(HISTORY, true))) {
 90              w.write(name + "," + new Date() + "," + a.toCsv());
 91              w.newLine();
 92          }
 93      }
 94
 95      static void showHistory() {
 96          List<String> rows = new ArrayList<>();
 97          try (BufferedReader r = new BufferedReader(new FileReader(HISTORY))) {
 98              String line;
 99              while ((line = r.readLine()) != null) {
100                  StringTokenizer st = new StringTokenizer(line, ",");
```

```
                rows.add(String.format("%-10s %-28s F0 %6s  J %5s%%  S %5s%%  HNR %5s
%s",
                        st.nextToken(), st.nextToken(), st.nextToken(), st.nextToken(),
                        st.nextToken(), st.nextToken(), st.nextToken()));
            }
        } catch (FileNotFoundException e) {
            System.out.println("No history yet.");
            return;
        } catch (IOException e) {
            System.out.println("Cannot read history: " + e.getMessage());
            return;
        }
        printAll("Saved history:", rows);
    }

    static <T> void printAll(String title, List<T> items) {     // generic method
        System.out.println(title);
        for (T item : items) System.out.println("  " + item);
    }
}
```

A.2 audio/VoiceSource.java (package audio)

```
package audio;

import javax.sound.sampled.*;
import java.io.*;
import java.util.Random;

// A voice to be screened. Abstract: each kind of source (microphone, WAV file, demo)
// loads its samples differently, so load() is overridden in each subclass.
public abstract class VoiceSource {
    public static final float RATE = 44100f;   // samples per second
    protected double[] samples;                // values between -1 and +1
    protected float rate;

    protected VoiceSource(float rate) { this.rate = rate; }

    public abstract void load() throws VoiceException;
    public abstract String getName();

    public double[] getSamples() { return samples; }
    public float getRate() { return rate; }

    @Override
    public String toString() {                 // overrides Object.toString()
        return String.format("%s (%.1f s at %.0f Hz)", getName(), samples.length / rate,
rate);
    }

    // 16-bit little-endian bytes -> numbers between -1 and +1 (first channel only)
    protected static double[] toSamples(byte[] b, int channels) {
        double[] x = new double[b.length / (2 * channels)];
        for (int i = 0; i < x.length; i++) {
            int j = i * 2 * channels;
            x[i] = ((b[j + 1] << 8) | (b[j] & 0xFF)) / 32768.0;
        }
```

```
34          return x;
35      }
36
37      // ---------- user-defined exception ----------
38      public static class VoiceException extends Exception {
39          public VoiceException(String message) { super(message); }
40      }
41
42      // ---------- 1. live microphone ----------
43      public static class Microphone extends VoiceSource {
44          private final int seconds;
45
46          public Microphone() { this(5); }                    // constructor overloading
47          public Microphone(int seconds) { super(RATE); this.seconds = seconds; }
48
49          public String getName() { return "Microphone"; }
50
51          public void load() throws VoiceException {
52              try {
53                  AudioFormat fmt = new AudioFormat(RATE, 16, 1, true, false);
54                  TargetDataLine mic = AudioSystem.getTargetDataLine(fmt);
55                  mic.open(fmt);
56                  mic.start();
57                  byte[] buf = new byte[(int) RATE * 2 * seconds];
58                  for (int got = 0; got < buf.length; ) got += mic.read(buf, got, buf.length
- got);
59                  mic.close();
60                  AudioInputStream out = new AudioInputStream(new ByteArrayInputStream(buf),
fmt, buf.length / 2);
61                  AudioSystem.write(out, AudioFileFormat.Type.WAVE, new
File("recording.wav"));
62                  samples = toSamples(buf, 1);
63              } catch (LineUnavailableException | IOException | IllegalArgumentException e)
{
64                  throw new VoiceException("Microphone not available: " + e.getMessage());
65              }
66          }
67      }
68
69      // ---------- 2. WAV file (final class: cannot be extended) ----------
70      public static final class WavFile extends VoiceSource {
71          private final String path;
72
73          public WavFile(String path) { super(RATE); this.path = path; }
74
75          public String getName() { return "WAV " + new File(path).getName(); }
76
77          public void load() throws VoiceException {
78              try (AudioInputStream in = AudioSystem.getAudioInputStream(new File(path))) {
79                  AudioFormat f = in.getFormat();
80                  if (f.getSampleSizeInBits() != 16 || f.isBigEndian()) {
81                      throw new VoiceException("Please use a 16-bit PCM WAV file");
82                  }
83                  rate = f.getSampleRate();
84                  samples = toSamples(in.readAllBytes(), f.getChannels());
85              } catch (UnsupportedAudioFileException | IOException e) {
86                  throw new VoiceException("Cannot read " + path + ": " + e.getMessage());
87              }
```

```
 88          }
 89      }
 90
 91      // ---------- 3. artificial 3-second "aah" at 140 Hz (demo without a microphone)
----------
 92      public static class Demo extends VoiceSource {
 93          private final boolean shaky;
 94
 95          public Demo() { this(false); }
 96          public Demo(boolean shaky) { super(RATE); this.shaky = shaky; }
 97
 98          public String getName() { return shaky ? "Demo shaky voice" : "Demo steady voice";
}
 99
100          public void load() {
101              Random r = new Random(7);
102              double jit = shaky ? 0.025 : 0.004;     // variation in cycle length
103              double shim = shaky ? 0.12 : 0.02;      // variation in cycle loudness
104              double noise = shaky ? 0.05 : 0.005;    // breathiness
105              samples = new double[(int) (3 * RATE)];
106              for (int start = 0; start < samples.length; ) {
107                  int n = (int) Math.round(RATE / 140 * (1 + jit * r.nextGaussian()));
108                  double amp = 0.5 * (1 + shim * r.nextGaussian());
109                  for (int k = 0; k < n && start + k < samples.length; k++) {
110                      double ph = 2 * Math.PI * k / n;
111                      samples[start + k] = amp * (Math.sin(ph) + 0.5 * Math.sin(2 * ph) +
0.25 * Math.sin(3 * ph))
112                              + noise * r.nextGaussian();
113                  }
114                  start += n;
115              }
116          }
117      }
118  }
```

A.3 analysis/PitchTracker.java (package analysis)

```
 1  package analysis;
 2
 3  import audio.VoiceSource;
 4  import audio.VoiceSource.VoiceException;
 5  import java.util.*;
 6
 7  // Finds every vocal cycle in a sustained "aah": how long it lasts and how high its peak
is.
 8  public class PitchTracker {
 9      final List<Double> periods = new ArrayList<>();   // cycle lengths in seconds
10      final List<Double> peaks = new ArrayList<>();     // cycle peak heights
11      double similarity;                                // how alike neighbouring cycles are
(0-1)
12
13      public PitchTracker(VoiceSource voice) throws VoiceException {
14          double[] all = voice.getSamples();
15          float rate = voice.getRate();
16          if (all.length < 2 * rate) throw new VoiceException("Too short. Record at least 2
seconds.");
17
```

```
        // 1. keep the steady middle (drop first and last 20 %) and remove the DC offset
        int from = all.length / 5;
        double[] x = Arrays.copyOfRange(all, from, all.length - from);
        double mean = 0, power = 0;
        for (double v : x) mean += v / x.length;
        for (int i = 0; i < x.length; i++) {
            x[i] -= mean;
            power += x[i] * x[i];
        }
        if (Math.sqrt(power / x.length) < 0.01) throw new VoiceException("Too quiet. Say
'aah' louder.");

        // 2. rough period = the lag (75-500 Hz) where the whole voice best matches itself
        int minLag = (int) (rate / 500), maxLag = (int) (rate / 75), best = minLag, len =
x.length - maxLag;
        for (int lag = minLag; lag <= maxLag; lag++) {
            if (corr(x, 0, lag, len) > corr(x, 0, best, len)) best = lag;
        }

        // 3. slide each cycle forward until it best matches the next one: that slide is
its length
        double t = 0, period = best, sum = 0;
        while (t + 2.5 * period < x.length) {
            int pos = (int) Math.round(t), n = (int) period, s = (int) (0.8 * period);
            double b = corr(x, pos, pos + s, n);
            for (int k = s + 1; k <= 1.2 * period; k++) {
                double c = corr(x, pos, pos + k, n);
                if (c > b) { b = c; s = k; }
            }
            double a = corr(x, pos, pos + s - 1, n), c = corr(x, pos, pos + s + 1, n), d =
a - 2 * b + c;
            period = d < 0 ? s + 0.5 * (a - c) / d : s;           // parabola through the
best match
            period = Math.max(minLag, Math.min(maxLag, period));   // stay inside 75-500 Hz
            double top = 0;
            for (int k = pos; k < pos + n; k++) top = Math.max(top, x[k]);
            periods.add(period / rate);
            peaks.add(top);
            sum += b;
            t += period;
        }
        if (periods.size() < 10) throw new VoiceException("No steady voice. Hold one 'aah'
for 3-5 s.");
        similarity = sum / periods.size();
        if (similarity < 0.5) throw new VoiceException("No clear voice found (only noise).
Try a quiet room.");
    }

    // how alike two pieces of the signal are: 1 = identical, 0 = unrelated
    static double corr(double[] x, int a, int b, int len) {
        double xy = 0, xx = 0, yy = 0;
        for (int i = 0; i < len; i++) {
            xy += x[a + i] * x[b + i];
            xx += x[a + i] * x[a + i];
            yy += x[b + i] * x[b + i];
        }
        return xy / Math.sqrt(xx * yy);
    }
```

```
69  }
```

A.4 analysis/VoiceReport.java (package analysis)

```
 1  package analysis;
 2
 3  import audio.VoiceSource;
 4  import audio.VoiceSource.VoiceException;
 5  import java.util.*;
 6
 7  // Turns the cycles into F0, jitter, shimmer and HNR, compares them with reference limits
 8  // and builds the report. Runs on its own thread (Runnable); sortable by jitter
(Comparable).
 9  public class VoiceReport implements Runnable, Comparable<VoiceReport> {
10      public static final String F0 = "Pitch F0 (Hz)", JITTER = "Jitter (%)", SHIMMER =
"Shimmer (%)", HNR = "HNR (dB)";
11      public static final double JITTER_LIMIT = 1.04, SHIMMER_LIMIT = 3.81, HNR_LIMIT = 20.0;
12
13      private final VoiceSource voice;
14      private final Map<String, Double> features = new LinkedHashMap<>();
15      private String error;
16      private boolean done;
17
18      public VoiceReport(VoiceSource voice) { this.voice = voice; }
19
20      public void run() {
21          try {
22              PitchTracker p = new PitchTracker(voice);
23              features.put(F0, 1 / mean(p.periods));
24              features.put(JITTER, 100 * change(p.periods) / mean(p.periods));
25              features.put(SHIMMER, 100 * change(p.peaks) / mean(p.peaks));
26              features.put(HNR, 10 * Math.log10(p.similarity / (1 - p.similarity)));
27          } catch (VoiceException e) {
28              error = e.getMessage();
29          } catch (RuntimeException e) {
30              error = "Analysis failed: " + e;
31          } finally {
32              finish();
33          }
34      }
35
36      private synchronized void finish() { done = true; }
37      public synchronized boolean isDone() { return done; }
38
39      static double mean(List<Double> v) {
40          double s = 0;
41          for (double d : v) s += d;
42          return s / v.size();
43      }
44
45      static double change(List<Double> v) {       // average difference between neighbours
46          double s = 0;
47          for (int i = 1; i < v.size(); i++) s += Math.abs(v.get(i) - v.get(i - 1));
48          return s / (v.size() - 1);
49      }
50
51      public double get(String name) { return features.get(name); }
```

```
52
53      public int markers() {                      // measures outside their limit (0-3)
54          int m = 0;
55          if (get(JITTER) > JITTER_LIMIT) m++;
56          if (get(SHIMMER) > SHIMMER_LIMIT) m++;
57          if (get(HNR) < HNR_LIMIT) m++;
58          return m;
59      }
60
61      public boolean needsFurtherCheck() { return markers() >= 2; }
62
63      public String report() throws VoiceException {
64          if (error != null) throw new VoiceException(error);
65          StringBuilder sb = new StringBuilder("Voice : " + voice + "\n");
66          for (Map.Entry<String, Double> e : features.entrySet()) {
67              sb.append(String.format("%-14s: %7.2f%n", e.getKey(), e.getValue()));
68          }
69          sb.append("Limits: jitter <= 1.04 %, shimmer <= 3.81 %, HNR >= 20 dB\n");
70          sb.append("Markers outside limits: " + markers() + " of 3\nRESULT: ");
71          sb.append(needsFurtherCheck() ? "FURTHER CHECK SUGGESTED (see a neurologist)" : "NO
CONCERN INDICATED");
72          return sb.toString();
73      }
74
75      public String toCsv() {
76          return String.format("%.1f,%.2f,%.2f,%.1f,%s", get(F0), get(JITTER), get(SHIMMER),
get(HNR),
77                  needsFurtherCheck() ? "CHECK" : "OK");
78      }
79
80      public int compareTo(VoiceReport other) {   // steadier voice (lower jitter) first
81          return Double.compare(get(JITTER), other.get(JITTER));
82      }
83
84      @Override
85      public String toString() {
86          return String.format("%-18s jitter %5.2f %%  shimmer %5.2f %%  HNR %4.1f dB -> %s",
87                  voice.getName(), get(JITTER), get(SHIMMER), get(HNR), needsFurtherCheck() ?
"CHECK" : "OK");
88      }
89  }
```

APPENDIX B: USER MANUAL

1. Install JDK 11 or newer and check with `java -version`.
2. Open a command prompt in the `exam-version` folder (it contains `Main.java`, `audio` and `analysis`).
3. Compile: `javac Main.java audio/*.java analysis/*.java` (on Windows: `audio\*.java analysis\*.java`, or double-click `run.bat`).
4. Run: `java Main` and type the patient's name.
5. For a live test choose **1**, take a breath and say a steady "aaah" at normal loudness for the whole 5 seconds, 20–30 cm from the microphone, in a quiet room. The recording is also saved as `recording.wav`.
6. To analyse an existing recording choose **2** and type the path of a 16-bit WAV file of at least 2 seconds. Phone recordings can be converted to WAV with any audio editor (for example Audacity: File → Export → WAV, signed 16-bit PCM).
7. To demonstrate without a microphone choose **3** (steady voice) or **4** (shaky voice).
8. Choose **5** to compare all voices analysed in this run, and **6** to see every saved result from `history.csv`.
9. Choose **0** to exit. Delete `history.csv` to clear the history.

If a check fails: "Too quiet" — speak louder or move closer; "No steady voice" — hold one vowel without stopping; "No clear voice found" — move to a quieter room; "Microphone not available" — allow microphone access for the terminal in the operating system settings, or use options 2–4.

APPENDIX C: TEAM CONTRIBUTION

Table C.1: *Work division by package*

Member	Roll number	Web prototype	Console version
Pranav	160125749063	Signal processing: audio and dsp packages (resampling, pitch detectors, pulse tracking)	Package audio (VoiceSource and subclasses); report
Bhanuteja	160125749048	Features and ML: features, ml, training and validation packages	Package analysis (PitchTracker, VoiceReport); accuracy study
Md. Ali	160125749055	Application: Main, server, db, session, storage, analysis packages and the website; live demo	Main class (menu, thread, history); test cases